ULM UNIVERSITY OF APPLIED SCIENCES
FACULTY OF COMPUTER SCIENCE

Thesis Title

A thesis submitted in partial fulfillment of the requirements for the degree of
Master of Science in Computer Science

by
Mohamad Arabi
Computer Science
Faculty of Computer Science, Ulm University of Applied Sciences, 2026

April
2026

Supervised by
Prof. Dr. Traub
Prof. Dr. [Co-supervisor]

Ulm, 2026

Statement

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Use of AI

AI tools were used to polish the text and improve the tone of this thesis. The ideas, design, implementation, and evaluation are my own.

Mohamad Arabi
Signature

.....

Date: 15 April, 2026

Abstract

Security operations teams are flooded with alerts, and most SIEM platforms still leave triage and response to human analysts. This thesis presents a generic SIEM reaction plugin that adds a reasoning layer on top of normal SIEM output. Alerts are normalized, enriched, scored with an explainable risk model, and passed to a planner that produces structured response plans. Each plan is checked by policy rules before any action is executed, and every step is recorded in an audit log.

The system is implemented as a single runnable project with a reasoning-focused interface that shows the full chain: raw alert, normalized alert, score and evidence, plan, policy decision, and execution result. The evaluation uses synthetic and public security datasets and shows strong planning accuracy with consistent, explainable outputs. The result is a practical foundation for moving from rule-heavy SIEM workflows to reasoning-driven response that is auditable, safe, and easy to extend.

Summary

Security operations centres face a structural gap between detection and response. SIEM platforms collect, correlate, and surface alerts—but what happens next remains a manual decision. Commercial SOAR tools execute pre-scripted playbooks, which break down on novel attack patterns and cannot reason about threat severity or asset criticality.

This thesis fills that gap with an intelligent reaction layer that sits downstream of a SIEM and upstream of any action. The system takes a normalised alert, enriches it with asset, identity, and threat-intelligence context, produces an explainable threat assessment, generates a structured response plan, and passes every proposed action through a deterministic policy engine before execution. All decisions are recorded in a structured audit log linking each action to its evidence.

The architecture is a modular monolith: a C# core engine enforces the orchestration pipeline and safety constraints, while a Python intelligence service provides scoring and planning via HTTP. The threat scorer combines TF-IDF logistic regression with contextual enrichment boosts. The planner is a custom 270K-parameter multi-head neural network that classifies strategy, predicts action sets, and regresses priority— resolving entity parameters deterministically rather than generating them, eliminating the hallucination risk that made LLM-based planning unsafe for live execution.

The planner achieves 96.4% strategy accuracy and 97.2% action F1 in end-to-end pipeline evaluation—matching a 250-million-parameter fine-tuned seq2seq model at under 5 milliseconds inference time. The policy engine guarantees that no model error can cause a hard-denied action to execute. The full system runs on a single development machine and exposes every stage of the decision pipeline for analyst review.

Acknowledgment

I would like to express my deepest appreciation to all those who contributed to the successful completion of my master's thesis. Foremost, I extend my heartfelt appreciation to my esteemed advisor, Prof. Dr.-Ing. Traub. His guidance, insights, counsel, and mentorship were instrumental in shaping this research.

I also thank Prof. [Second Examiner], who graciously served as the second examiner. Lastly, I would like to thank all individuals and resources, too numerous to name, who contributed indirectly through their research, books, articles, and online sources.

Mohamad Arabi
Department of Computer Science
Technische Hochschule Ulm
Ulm, Germany
15 April, 2026

Table of Contents

List of Figures

List of Algorithms

List of Abbreviations

SIEM	**S**ecurity **I**nformation and **E**vent **M**anagement
SOAR	**S**ecurity **O**rchestration, **A**utomation, and **R**esponse
SOC	**S**ecurity **O**perations **C**enter
EDR	**E**ndpoint **D**etection and **R**esponse
XDR	e**X**tended **D**etection and **R**esponse
IOC	**I**ndicator **o**f **C**ompromise
TTP	**T**actics, **T**echniques, and **P**rocedures
ATT&CK	Adversarial **T**actics, **T**echniques, and **C**ommon **K**nowledge
LLM	**L**arge **L**anguage **M**odel
ML	**M**achine **L**earning
JSON	**J**ava**S**cript **O**bject **N**otation
API	**A**pplication **P**rogramming **I**nterface
STIX	**S**tructured **T**hreat **I**nformation e**X**pression
TAXII	**T**rusted **A**utomated e**X**change of **I**ndicator **I**nformation

List of Symbols

R	risk score (overall)
b	bias term in risk model
w_i	feature weight i
x_i	feature value i
m	context multiplier
c	confidence (0–1)
S	severity (0–100)
P	response priority (0–100)

Introduction

1.1 Motivation

Security Operations Centers (SOCs) rely on SIEM platforms to collect and centralise security logs from across the infrastructure. [1] Theoretically, this gives analysts a complete view of what is happening in the environment. In practice, they deal with a constant flow of alerts—many of them noisy, duplicated, or low quality—and still need to decide which ones actually signal a real threat. [2]

Alert queues accumulate faster than teams can clear them. Response time increases, and there is always the pressure that something important will be missed. This is alert fatigue: the volume of noise becomes harder to manage than the actual threats. [3]

Consider a simple example. Several failed login attempts could mean a brute-force attack—or a misconfigured script. A human analyst can tell the difference, but only if the right context is available and presented clearly. The problem is not just detection. It is reasoning, prioritization, and deciding what to do next.

1.2 Problem Statement

Most SIEM environments rely on rule-based detection. Over time, rule sets grow, overlap, and produce redundant alerts, making it harder to find what actually needs attention. [4] Even where automation exists, SOC workflows still depend heavily on human judgment for response decisions. [3]

The gap is clear: SIEM platforms are good at collecting and detecting, but they offer little support for reasoning about what to do after an alert fires. [2] The system generates an alert, but the plan for responding to it—and any feedback from how that response went—stays entirely with the analyst.

This thesis proposes a response reasoning layer that sits downstream of alert normalization. It evaluates alerts, produces explainable risk scores, generates structured response plans, applies policy constraints before any action is taken, and maintains an audit trail of every decision.

1.3 Research Objectives

- Design a layered architecture that separates orchestration from intelligence and allows detection models to be replaced without disrupting the surrounding SIEM pipeline.
- Implement an explainable risk scoring engine that uses evidence-based features along with contextual information and offers an explanation for each score.
- Create a structured planning module that offers response plans, policy conformance for those plans, and rollbacks.
- Integrate analyst feedback and investigation traces so that the system can learn from past investigations and suggest useful next steps.
- Demonstrate the approach through an end-to-end prototype that includes a runnable system and an interface focused on reasoning and investigation support.

1.4 Thesis Structure

Chapter 2 reviews related work in SIEM systems, alert correlation, and response automation. Chapter 3 defines the system requirements. Chapters 4 and 5 cover the system design and implementation. Chapter 6 describes the dataset pipeline and model training. Chapter 7 evaluates the system. Chapter 8 discusses the results and limitations. Chapter 9 concludes and outlines future directions.

Background and Related Work

2.1 Cybersecurity Monitoring and Security Operations Centers

Modern organisations generate large volumes of digital events across their infrastructure—network flows, authentication attempts, file system changes, process executions—from servers, workstations, cloud instances, and IoT devices. The central mission of a *Security Operations Center* (SOC) is to monitor this activity continuously, triage suspected incidents, and coordinate the response.

The procedural foundation for SOC operations comes from the NIST Computer Security Incident Handling Guide [5], which defines a four-phase lifecycle: Preparation, Detection and Analysis, Containment/Eradication/Recovery, and Post-Incident Activity. The guide notes that detection and analysis is the most labor-intensive phase. Tilbury and Flowerday [3] confirm this in practice—SOC analysts operate under persistent workload pressure from alert volumes that exceed what any team can manually triage within a shift. Their study argues for augmentation over replacement: systems that absorb the mechanical triage burden so analysts can focus on the cases that genuinely need human judgment.

2.2 Security Information and Event Management (SIEM)

2.2.1 Log Collection, Normalization, and Correlation

A SIEM platform is the core technology of a SOC [1]. It collects log data from heterogeneous sources—operating systems, applications, network devices, security appliances—aggregates it centrally, and makes it queryable by analysts and automated correlation

engines. The normalization problem is harder than it looks: a `severity` field in a firewall log means the configured rule priority, while the same field in an antivirus alert means the malware's assessed danger level. Getting this wrong propagates errors into every downstream component.

Event correlation is how raw logs become meaningful alerts. A single failed SSH login is unremarkable; ten thousand from a single IP in thirty seconds is a brute-force attempt.

Valeur et al. [6] offer a structured model of the full alert correlation workflow, breaking it down into a sequence of discrete stages that each serve a distinct purpose: raw events are first normalised and deduplicated, then related alerts are fused into higher-level signals, correlated across time and source, filtered to reduce false positives, and finally grouped into coherent attack sessions. What this model makes clear is that what most practitioners call "alert correlation" is not one operation but several, and conflating them makes the system harder to reason about and harder to tune.

Time-window correlation rules—the workhorse of most commercial SIEMs—are computationally efficient and interpretable, but they require an expert to anticipate every relevant pattern. Rules that are too specific miss variants; rules that are too broad generate noise.

2.2.2 Alerting and Incident Response

The output of the correlation engine is an alert asserting that a pattern matches a known-bad template. An analyst must then assess its fidelity, severity, and appropriate response. In theory this is a structured workflow [5]; in practice, prioritization varies considerably across shifts and teams depending on workload, experience, and fatigue [3]. Inconsistent triage is one of the core motivations for machine-assisted scoring.

2.3 Limitations of Current SIEM Systems

2.3.1 Rule-Based Detection and Alert Overload

The fundamental limitation of rule-based correlation is that rules only encode patterns an expert anticipated—they say nothing about patterns that were not. *Living-off-the-land* techniques deliberately use legitimate system tools like PowerShell, WMI, and certutil to carry out malicious activity in a way that evades signature-based detection [7]. Sarhan et al. [4] document how rule sets accumulate over time without retirement, generating redundant and overlapping false positives that are difficult to reason about.

The consequence is alert fatigue. Ban et al. [2] report false-positive rates exceeding 90% in some deployments—nine benign alerts for every genuine threat. The base-rate fallacy, formalised by Axelsson [8], explains why this is structurally difficult to avoid: even a detector with 99% sensitivity and 99% specificity produces roughly one false positive per true positive when the true attack rate is 1-in-10,000 events. Acceptable false-positive performance requires specificity approaching 99.99%+, a bar that most deployed systems—rule-based or ML-based—rarely sustain under distribution shift.

2.3.2 Dependence on Human Analysts

The deeper structural limitation is that SIEM effectiveness is bounded by analyst availability and skill. Tilbury and Flowerday [3] document that skilled analysts are globally scarce, SOC turnover is high due to burnout, and the knowledge required to interpret alerts is tacit and difficult to transfer. Automation cannot replace analysts—the authors are explicit that high-stakes decisions must remain human-supervised—but it should absorb the mechanical parts of the triage workflow so analysts can focus where their judgment is genuinely needed.

2.4 Machine Learning Approaches to Intrusion Detection

Rule-based detection limitations have motivated substantial research into ML-based intrusion detection. The two dominant paradigms are anomaly-based detection—training a model on normal behaviour and flagging deviations—and signature-based detection, which applies classifiers trained on labelled attack examples. Garcia-Teodoro et al. [9] survey the anomaly field comprehensively, finding that ML-based approaches achieve high detection rates for novel attacks but produce elevated false-positive rates because the space of "abnormal but benign" events is vast. Signature-based approaches achieve high precision when attack samples are available but face the same generalisation problem as rule-based systems: variants and zero-days fall outside the training distribution.

Sommer and Paxson [10] provide an influential critical analysis of ML for network intrusion detection, identifying challenges absent in typical ML benchmarks: extreme class imbalance, distribution shift as adversaries adapt, high false-positive costs, and opacity of model decisions. Their critique has aged well. It anticipates the explainability requirement this thesis addresses directly—every score produced must be accompanied by evidence and reasoning an analyst can evaluate. More recent deep learning approaches [11–13] show promise on benchmarks but amplify this interpretability concern: the deeper the model, the harder it is to explain a specific flagged alert [3, 10].

2.5 Automated and Intelligent Incident Response

Academic work on the response side of the pipeline is thinner than on detection. Ban et al. [2] describe an AI-assisted SIEM framework that automatically prioritizes and summarises alerts, demonstrating meaningful workload reductions, though the response component remains recommendation-only—a human still executes every action. This pattern is common: automated response with actual execution is rare because of the safety concerns it raises [3, 5].

Commercial *Security Orchestration, Automation, and Response* (SOAR) platforms—Palo Alto XSOAR, Splunk SOAR, IBM Security SOAR—take a playbook-scripted approach. An expert defines workflows in advance; the platform executes them on trigger [5]. This works well for high-frequency, well-understood incident types. It breaks down on novel patterns or hybrid scenarios where no playbook matches, and it provides no mechanism to learn from outcomes [3]. The MITRE ATT&CK framework [7] has become a common reference for grounding alert types to adversary behaviour and appropriate response actions, and this thesis uses it as one basis for the response planning training data.

2.6 Research Gap

The literature reveals a consistent pattern: detection is more mature than response, and the triage layer between them remains heavily human-dependent. This thesis targets three specific gaps.

First, existing SIEM platforms are passive—they surface alerts but do not reason about them, produce response plans, or take any action. The system built here starts where the SIEM stops: it accepts a normalised alert and adds learned threat assessment, strategy-level response planning, policy-gated action execution, and a full audit trail. It is designed to sit downstream of a SIEM, not to replace one.

Second, commercial SOAR platforms execute pre-authored playbooks. They cannot reason about an alert they have never seen, weigh a threat's severity against the impact of the response, or adapt when the alert falls between playbook categories. This thesis replaces the static playbook with a trained model that infers response strategy from the observed alert context, backed by a deterministic policy layer that enforces hard safety constraints independently of model quality. The result achieves over 96% strategy accuracy at under 5 milliseconds of inference time, without any generative component that could hallucinate unsafe parameter values.

System Requirements

3.1 Functional Requirements

The system ingests alerts from external SIEM sources, normalises them to a common schema, enriches them with contextual data, and produces ranked incident candidates. For each incident the scoring component generates an explainable risk score and confidence value, accompanied by reason codes and context multipliers. The planning component then produces a structured response plan—strategy, priority, action set, and rollback steps—which is validated against explicit policy rules before any action is dispatched. The execution stage records action outcomes and errors.

Analyst feedback is a first-class input. An analyst can mark an incident as true positive, false positive, or needs more information, and optionally override severity. Feedback events are recorded in an append-only log and used to update model weights in a bounded, auditable way. Investigation traces are tracked and used to recommend next steps. A user interface exposes raw alerts, normalised alerts, scoring explanations, plans, policy checks, execution status, and audit logs.

3.2 Non-Functional Requirements

The architecture is modular so that intelligence models can be swapped without touching orchestration code. The interface between layers is explicit and versioned. The system must run as a single project to facilitate demonstrations and repeated experiments. Explanations are human-readable and consistent across alerts to allow direct comparison.

There is a clear separation between domain logic, application services, infrastructure, and the UI. Structured logging throughout facilitates audits and retrospective analysis.

3.3 Security and Safety Requirements

Automation is bounded by design. All planner actions are subject to policy rules—allow/deny lists, confidence thresholds, asset criticality limits. High-impact actions require approval gates, and each automated action has a corresponding rollback. The execution layer validates parameters, enforces entity-presence checks, and refuses to execute when required context is missing or policy checks fail. Rate limits and idempotency safeguards prevent repeated execution under noisy alert conditions.

If confidence is too low or policy approval is not granted, the system falls back to recommendation-only behaviour. Model risk is controlled through bounded, logged weight updates. Model output is traceable after updates, and the system supports drift detection and periodic offline recalibration. Connectors and executors operate under least-privilege access, and sensitive fields are minimised or redacted where possible. All policy evaluations and execution outcomes are logged with timestamps and actor identity.

System Design and Architecture

4.1 Design Goals

Before any implementation decision was made, a set of concrete design goals was established to guide the architecture. These goals come from the functional and non-functional requirements described in Chapter 3 and reflect the realities of deploying automated response in a security operations context.

Explainability over opacity. Every decision the system makes—a severity score, a chosen strategy, an approved or denied action—must be traceable to a specific reason that a human analyst can read and verify. Treating the system as a black box is simply not acceptable here: even the machine-learning components must surface evidence strings, hypothesis text, and per-action rationale rather than returning bare predictions. Explainability is not a feature; it is a hard constraint on the choice of models and output formats.

Safety before speed. The cost of a false positive in autonomous threat response—blocking a legitimate IP, isolating a production server, disabling an active user account—can be severe and immediate. The architecture must guarantee that no high-impact action is ever taken without a multi-stage validation chain (intelligence assessment, structured planning, policy evaluation) and, where the risk is above a threshold, explicit human approval. Throughput and latency are secondary to correctness.

Modularity and replaceability. Security tooling evolves quickly. The threat scoring model that works well today may be superseded within months. Any component—scorer, planner, SIEM connector, action handler—must be replaceable without cascading changes to the rest of the system. This is achieved through explicit interfaces at every layer boundary.

Operability as a single project. The system must be runnable end-to-end on a single development machine for experimentation and demonstration. In practice, this means avoiding hard dependencies on cloud services, managed databases, or always-on external infrastructure. Everything that is needed must either be embedded or configurable through environment variables.

Reproducibility. Given the same input alert and the same configuration, the system must produce the same output. This is non-trivial when machine-learning models, LLM-based components, and external enrichment lookups are all in play. The architecture addresses this through response caching, deterministic post-processing pipelines, and the ability to replay any alert from a stored demo trace.

4.2 Key Architectural Decisions

Several structural decisions shape the entire system. They are worth explaining before describing the components they produce.

4.2.1 Modular Monolith over Multi-Agent Architecture

The first and most fundamental choice was the overall *architectural style*. An alternative that was seriously considered—and ultimately rejected—was a *multi-agent architecture*, where autonomous software agents collaborate to detect and respond to threats. In a multi-agent SOAR, distinct agents might handle ingestion, enrichment, scoring, planning, and execution, communicating asynchronously through a shared message bus or blackboard. Each agent would have its own perception loop, internal state, and decision logic.

Multi-agent architectures are appealing in theory because they mirror the way human SOC teams actually work—different analysts specialize in different tasks—and because they seem well-suited to parallel workloads. They are also the dominant framing in much of the AI-oriented security automation literature, where the vocabulary of "agents," "goals," and "environments" maps naturally onto reinforcement learning formulations. In practice, though, closer examination showed that multi-agent design would have introduced significant costs with limited benefit for this system's goals.

Accountability and auditability. When a response action is taken by a loosely coupled multi-agent system, attributing that decision to a specific chain of reasoning is difficult. Messages between agents are transient, the causal chain crosses multiple independent processes, and reconstructing exactly why the system acted as it did requires

correlation across distributed logs. In a monolith where every decision traverses a single, linear, in-process call chain, this is straightforward—the whole chain can be recorded in full.

Safety guarantees. A policy layer that must co-ordinate with autonomous agents over a message bus cannot make the synchronous, hard guarantees the safety requirements demand. An agent that acts before policy evaluation completes, or that bypasses a policy message due to a queue timeout, creates a window in which unsafe actions can execute. The synchronous, in-process policy engine chosen here makes that window structurally impossible: no action can proceed past the policy evaluation call.

Operational complexity. Multi-agent systems bring distributed-systems failure modes—network partitions, agent crashes, message ordering, duplicate delivery, split-brain state—that must be handled on top of the domain logic itself. For a research prototype that must run on a single machine and be demonstrable to non-infrastructure audiences, this complexity adds nothing.

Emergent behavior. Perhaps the most serious concern is that autonomous agents interacting over a shared environment can exhibit emergent behaviors that are difficult to predict, reproduce, or explain. In a security context, emergent behavior in a response system is an unacceptable risk.

The architecture chosen instead is a **modular monolith**: a single cohesive process (the core engine) where all orchestration stages execute synchronously in a well-defined sequence, with clear interfaces between components. The word "monolith" here does not imply a tangled codebase—each component is encapsulated behind an interface and can be swapped independently—but it does mean that control flow is explicit, predictable, and fully traceable.

The only intentional process boundary in the system is the HTTP boundary between the core engine and the intelligence service. This boundary exists not for autonomy or parallelism but for language-ecosystem isolation: the C# runtime and the Python runtime are genuinely different environments with different strengths, and keeping them separate avoids compromising either. The intelligence service is not an autonomous agent—it is a stateless function-as-a-service that the orchestrator calls synchronously and whose output the orchestrator validates before acting on it. Authority always rests with the core engine; the intelligence service provides recommendations, not decisions.

The reason for this choice comes down to a practical principle: in a system that takes real-world actions in a production security environment, human-interpretable control flow and hard safety guarantees matter more than the theoretical scalability benefits of autonomous agent autonomy.

4.2.2 Two-Process Boundary: C# Orchestration and Python Intelligence

The most consequential architectural decision is to implement the orchestration layer in C# (.NET 9) and the intelligence layer in Python 3.11, with the two communicating over a local HTTP boundary.

This separation was not driven by language preference but by genuine domain fit. The orchestration layer handles control-plane work: managing state across alert cycles, dispatching concurrent actions, enforcing policy constraints, writing audit records, and maintaining SIEM connectivity. These tasks benefit from .NET's strong typing, high-performance async I/O, structured concurrency, and its mature ecosystem for building long-running services. The intelligence layer, by contrast, handles model inference and training: running PyTorch models, calling scikit-learn pipelines, wrapping Hugging Face transformers, and querying local LLMs via Ollama. Python is the unambiguous standard for this work.

Hosting both in a single process—via Python.NET, a C Python binding, or a gRPC bridge inside the same runtime—would mean either compromising the orchestrator's performance characteristics for the sake of the ML code, or fighting against whichever runtime was forced to be the guest. The HTTP boundary is a clean contract: the C# side makes a POST request and receives a JSON response; the Python side is free to load gigabyte-scale models, use the GIL, and import whatever it needs without affecting the orchestrator's latency.

The boundary also provides an important operational benefit: either process can be restarted, upgraded, or replaced independently. A new model checkpoint can be hot-swapped in the intelligence service without restarting the core engine, and the core engine can be recompiled and updated without disturbing a running model server.

4.2.3 Interface-Driven Design in the Core Engine

Every major component in the C# orchestrator is expressed as an interface: `ISiemConnector`, `INormalizationPipeline`, `IThreatScorer`, `IPlanner`, `IExecutionPipeline`, `IAuditLogger`, and `ICaseManager`. The `AgentOrchestrator` class depends only on these interfaces; it has no knowledge of the concrete implementations injected into it.

This is not boilerplate. Interface-driven design enables two critical capabilities. First, testing: the entire orchestrator can be exercised with lightweight in-memory stubs for every external dependency, making it possible to run thousands of alert scenarios in

milliseconds without a live SIEM, HTTP service, or database. Second, replaceability: switching from the HTTP-based `HttpThreatScorerClient` to a hypothetical embedded scorer requires changing a single dependency-injection registration, not modifying the orchestrator itself.

The interfaces are also the system's version contract. When the Python intelligence service evolves its response schema, only the HTTP client adapter needs to be updated; the rest of the orchestration code is unaffected.

4.2.4 Policy as a Local, Synchronous, Hard Layer

A critical design decision is that the policy engine lives entirely within the core engine—it is not a microservice, not a call to the intelligence service, and not a model. It is synchronous, deterministic, and executes in memory with no external dependencies.

The reason for this comes directly from the safety requirement. If policy were implemented as a remote call, a network failure, a service restart, or a latency spike could result in actions being executed without policy evaluation. By keeping policy local and synchronous, the system guarantees that no action can bypass policy under any operational condition short of a process crash. Hard denials are evaluated before approval gating, so entity-presence violations are caught immediately rather than being queued for human review that can never be resolved.

4.2.5 Structured Prediction over Generative Planning

An early design explored using large language models to generate response plans as free-form JSON. This approach was abandoned after discovering two fundamental problems. First, sentence-piece tokenisers used by encoder-decoder models cannot reliably generate curly-brace characters, causing structured output to fail at inference time. Second, and more seriously, LLMs hallucinate entity values: they generate plausible-looking host IDs, IP addresses, and file hashes that do not correspond to any entity in the actual alert, producing plans that would target the wrong resources.

The solution was to decompose planning into *structured prediction*: the model classifies strategy, predicts a binary action set, and regresses priority. Entity parameters are then resolved deterministically from the enriched alert context. This eliminates hallucination entirely and reduces the planning problem to one that a small custom neural network can solve with over 96% accuracy at a fraction of the inference cost of an LLM.

4.2.6 Reasoning and Explainability Design

Explainability is woven into the data model rather than added as an afterthought. Every major data structure carries human-readable explanation fields: `ThreatAssessment` includes a `hypothesis` and an `evidence` list; `DecisionPlan` includes a `rationale` list and per-action `rationale` strings; `PolicyDecision` includes per-action `reasons`. These fields are populated by the intelligence service and the policy engine respectively, recorded in the audit log verbatim, and displayed on the console during processing.

What this means in practice is that no explanation is synthesized after the fact—what the analyst sees is the same text that drove the decision, not a post-hoc reconstruction. The consequence for model design is that any scorer or planner that cannot produce natural-language rationale is a poor fit for this system, regardless of its accuracy.

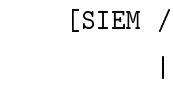
4.2.7 Safety and Fallback Mechanisms

The system is designed to degrade gracefully under failure, not to stop. Each of the pipeline stages that can fail—normalization, scoring, planning, policy, execution—is wrapped in an isolated try-catch boundary. A failure in any stage produces a structured error record in the audit log and a graceful outcome, but does not propagate to the caller or interfere with other alerts being processed concurrently.

The intelligence service applies the same principle through its scorer and planner hierarchies. If the primary classifier scorer fails, the hybrid scorer falls back to the Ollama LLM; if Ollama is unavailable, a deterministic rule scorer is used. The planner similarly falls back to a rule-based planner if the neural network inference fails. The system never surfaces an empty result—there is always a response, even if it is a conservative one. Security teams would rather receive a cautious but well-formed response than an error that silently drops an alert.

4.3 System Overview

With the design decisions established, the architecture resolves into two cooperating processes and a linear alert-processing pipeline. The pipeline for each alert follows seven sequential stages:



```
      v
[Event Ingestion]      (pull or webhook, cursor-based pagination)
      |
      v
[Normalization & Enrichment]  (schema alignment, asset/identity/TI lookup)
      |
      v
[Threat Scoring]       (HTTP -> Intelligence Service /v1/score)
      |
      v
[Decision Planning]    (HTTP -> Intelligence Service /v1/plan)
      |
      v
[Policy Evaluation]    (local, synchronous, hard safety rules)
      |
      v
[Action Execution]     (approved actions dispatched; pending queued)
      |
      v
[Audit & Case Management]  (JSONL log, case database, demo trace)
```

The two-process boundary sits between the Policy Evaluation stage and the Threat Scoring / Decision Planning stages:

```
+---------------------------+   HTTP/JSON    +---------------------------+
|       Core Engine         | <-----------> |   Intelligence Service    |
|  (C# / .NET 9)            |  /v1/score    |   (Python / FastAPI)      |
|                           |  /v1/plan     |                           |
|  AgentOrchestrator        |               |  ClassifierScorer         |
|  PolicyEngine             |               |  CustomNNPlanner          |
|  ExecutionPipeline        |               |  ModelRegistry            |
|  AuditLogger              |               |  LruCache                 |
+---------------------------+               +---------------------------+
           |                                           |
  SIEM connectors                            intelligence/models/
  Action integrations                        (planner_nn/, classifier/)
```

4.4 Event Ingestion Layer

Every orchestration cycle begins with the ingestion of raw security events. Both ingestion modes are expressed through the same `ISiemConnector` interface, so downstream stages are unaware of which mode is active.

Polling mode is the default. The orchestrator calls `PullAlertsAsync` each cycle, passing a *cursor* that encodes the position reached in the last successful fetch. Cursor-based pagination is stateless from the SIEM's perspective and resilient to restarts: the cursor is preserved in memory and reused across cycles, so replay from the last checkpoint is automatic. When no cursor exists on first run, the engine falls back to a configurable `SinceUtc` timestamp.

Webhook mode is the alternative for SIEMs that push events. A lightweight ASP.NET Core endpoint accepts inbound HTTP POST requests and calls `AgentOrchestrator.HandleAlertAsync` directly. Both modes share all downstream stages; only the entry point differs.

The `RawAlert` type produced at this stage is intentionally minimal—it carries only the data the SIEM itself supplies: an identifier, a timestamp, a rule name, an alert type, the originating SIEM name, and an opaque `Properties` dictionary for vendor-specific fields. This prevents upstream schema variations from leaking into downstream components.

4.5 Normalization and Enrichment

Raw alerts must be translated into the system's canonical `EnrichedAlert` representation before any intelligence component can operate on them. This is the responsibility of the `INormalizationPipeline`, implemented as `BasicAlertValidator`.

Schema normalization maps vendor-specific field names to a stable internal schema. Alert types are canonicalized to one of five categories: `PortScanFromIp`, `BruteForceUser`, `MalwareHashOnHost`, `SuspiciousProcessOnHost`, and `BenignNoise`. Severity values from heterogeneous scales are rescaled to a uniform 0–100 integer.

Context enrichment augments each normalized alert with three independent lookups performed in parallel:

- **Asset context**: Given a host identifier or IP address, the enrichment layer queries an asset registry for the asset's `criticality` (1–5), its network `environment` (internal/DMZ/external), and privileged status. Criticality 5 assets trigger mandatory escalation regardless of the scorer's output.

- **Identity context**: If the alert references a username, the identity provider is queried for the user's privileged flag, department, and role. Privileged users carry higher inherent risk weight in both scoring and planning.
- **Threat intelligence**: IP addresses and file hashes are checked against a threat-intelligence feed, returning a `reputationScore` (0–100) and matching tags such as *known-scanner* or *c2-server*. A high TI score amplifies scorer confidence.

If any lookup fails, the enrichment stage records null for that context block rather than aborting—a deliberate fail-open design that prioritises pipeline continuity.

4.6 Threat Scoring Component

The enriched alert is submitted to the intelligence service's `/v1/score` endpoint. The returned `ThreatAssessment` contains a `severity` integer (0–100), a `confidence` float (0–1), a natural-language `hypothesis`, and a list of `evidence` strings.

The intelligence service scores alerts through a configurable hierarchy. In production the default is a **hybrid**: a TF-IDF + Logistic Regression classifier is the primary signal, an optional Ollama-hosted LLM provides secondary explanation when confidence is borderline, and a rule-based fallback ensures the pipeline never returns an empty score.

The `ClassifierScorer` represents alert fields as a bag-of-words TF-IDF vector concatenated with structured features (alert type, asset criticality, privileged flag, exposure zone). Post-processing applies calibrated confidence floors per alert type and boosts the final score when the enrichment context contains a high TI reputation or a criticality-5 asset.

The scorer is a read-only assessment—it provides evidence for the planning decision, not a trigger. The `hypothesis` and `evidence` fields are recorded verbatim in the audit log alongside every action taken, so analysts reviewing historical cases can reconstruct the system's reasoning exactly.

4.7 Decision Planning Component

Given an enriched alert and a threat assessment, the planner produces a `DecisionPlan` specifying a **strategy**, a **priority**, an ordered list of **planned actions**, and **rationale** strings. The strategy is one of five values:

- **Ignore**: The alert is assessed as benign noise; no action is taken.
- **NotifyOnly**: Low severity or confidence; notify the SOC and open a ticket.
- **Investigate**: Warrants investigation but no automated containment.
- **ContainAndCollect**: Active threat confirmed; containment plus forensics.
- **Contain**: Immediate high-confidence threat; containment is the priority.

The intelligence service exposes planning through `/v1/plan`. Multiple backends are supported via the model registry. In production the **CustomNNPlanner** is used—a lightweight multi-head MLP trained from scratch on 8,644 labeled examples.

The neural network decomposes planning into three parallel structured-prediction problems: strategy (5-class softmax), action set (multi-label sigmoid over 8 action types), and priority (sigmoid scaled to 0–100). A deterministic post-processing pass then resolves entity parameters from the enriched alert context, adds safe infrastructure actions that are always appropriate, and applies label sanitization to enforce coherence. This eliminates LLM hallucination entirely and reduces inference time to under 5 ms.

The model registry allows multiple named profiles to coexist. A caller can request a specific profile via the `X-Intel-Profile` HTTP header, enabling A/B evaluation or graceful rollback without any change to the core engine.

4.8 Policy and Safety Layer

The policy engine is the system's safety backstop. It operates entirely within the core engine—synchronous, deterministic, no external calls—and evaluates every planned action against hard constraints before dispatch. Each action is triaged into one of three categories:

- **Approved**: Passes all constraints; auto-executed.
- **Pending**: Valid but above the risk threshold; held for human approval.
- **Denied**: Violates a hard constraint; permanently blocked for this cycle.

Hard denials are evaluated first. They cover: actions requiring an entity (host ID, IP, username) that is absent from the enriched alert; irreversible actions on production assets below a severity threshold; and globally disabled action types. This ordering prevents a

pending action from being queued for human review when no human action could resolve the underlying constraint violation.

Threshold-based approval gating sits below hard denials. Actions above a configurable risk threshold require human approval even if all entity constraints are satisfied. Thresholds are tunable per environment. All policy decisions and their specific reasons are recorded in the audit log and displayed in the console during processing.

4.9 Action Execution and Response

Approved actions are dispatched by the `IExecutionPipeline`. Each `PlannedAction` carries a type (e.g., `BlockIp`, `IsolateHost`, `QuarantineFile`), typed parameters, a risk level, and an `expectedImpact` score for ordering within a plan.

Dry-run mode (the default) validates and logs every action with a `DryRun` status but makes no external calls. The audit trail produced in dry-run mode is structurally identical to that in live mode, so the transition to live execution requires no changes to reporting or observability tooling.

Live mode dispatches actions to real integration targets. Handlers are resolved from the `ActionCatalog`, invoked with a configurable timeout, and their results collected into an `ExecutionReport` recording each action's status: `Succeeded`, `Failed`, `Skipped`, or `DryRun`.

Rollback actions (`UnblockIp`, `UnisolateHost`, `EnableUser`) are included in every plan containing a corresponding containment action. They are stored in the plan's `rollbackActions` list and surfaced to analysts as reversibility documentation; they are not executed automatically.

After execution, the orchestrator calls `TryAckAsync` to update the SIEM's alert status. Alerts with pending actions are acknowledged as `InProgress`; fully resolved alerts are marked `Closed`.

4.10 Audit and Explainability Components

Every significant event is recorded by `IAuditLogger` as a structured `AuditEntry` in a newline-delimited JSON file. Each entry carries a correlation ID, a timestamp, a component name, an event type, a human-readable message, and a context-specific key-value dictionary. The log is append-only and designed for post-hoc querying.

Beyond the event log, the system writes a **demo trace** for each processed alert: a complete snapshot of the raw alert, enriched alert, threat assessment, decision plan, policy decision, and execution report. These traces are written to `demo_trace.jsonl` and serve two purposes. First, they enable full reproducibility—any alert can be replayed deterministically from its trace. Second, they power a demo user interface that visualises the complete decision pathway for stakeholder demonstrations.

The `ICaseManager` groups related alerts into persistent cases backed by a SQLite database. Cases accumulate assessments and plans across alerts, providing a unified timeline view for multi-stage attacks that would be invisible at the individual-alert level.

4.11 Cross-Cutting Concerns

Fault isolation. Each pipeline stage is wrapped in an independent try-catch boundary. A failure at any stage produces a structured error record and a graceful outcome, but does not propagate or affect other alerts being processed concurrently. Audit and case management failures are additionally isolated, since secondary concerns must never degrade alert handling throughput.

Observability. Each processed alert triggers `PrintDemoSummary`, which emits a structured, ANSI-colored console block showing the alert identity, enrichment context, assessment scores, plan strategy, and per-action policy decisions. At session end, `PrintExecutiveSummary` emits an aggregate table: total alerts processed, strategy breakdown, action counts by approval tier, and the highest-severity alert seen.

Response caching. The intelligence service maintains an LRU cache keyed on a hash of the alert content and the active model profile. Repeated submissions of the same alert—common when a SIEM re-delivers unacknowledged events—are served from cache without invoking the scorer or planner. Cache capacity defaults to 256 entries and is configurable via `INTEL_CACHE_SIZE`.

Configuration. All runtime parameters are expressed through environment variables loaded from a `.env` file at startup. Neither side requires file-system writes or database migrations for configuration changes, simplifying deployment in containerized environments and making exact configurations reproducible from a single `.env` snapshot.

Implementation

Building a system that reasons about security threats is, at its core, an exercise in managing complexity across layers that evolve at very different speeds. The orchestration layer must be rock-solid and deterministic — it cannot afford to crash when a model fails or a network call times out. The intelligence layer, on the other hand, needs to be fluid: models get retrained, thresholds get tuned, and entirely new backends can be swapped in overnight. Getting this boundary right was one of the first and most consequential decisions in the implementation.

This chapter walks through every major component of the system, explaining not just what was built but why each decision was made the way it was. The intent is to give a software engineer enough context to understand, reproduce, or extend any part of the implementation without needing to reverse-engineer intent from code alone.

5.1 Technology Stack

The system is split into two cooperating services with distinct runtime requirements, and the technology choices reflect that distinction honestly.

5.1.1 Orchestration Layer: .NET 9 / C#

The orchestration engine is written in C# and targets `net9.0`. C# was chosen because it provides strong static typing, high-performance async I/O, and a mature dependency injection container. These are not incidental features — they are the specific properties the orchestration layer needs. Strong typing catches interface contract violations at compile time. Async I/O allows concurrent enrichment lookups without thread-per-request overhead. The DI container wires the pipeline together with minimal ceremony and makes it trivial to swap any component.

The decision not to use a full web application framework like ASP.NET was deliberate. The engine runs as a console application, not a web server, because its job is to process alerts, not to serve HTTP requests. The webhook *listener* is a small HTTP utility used only in webhook mode — the engine remains a pipeline, not a service.

5.1.2 Intelligence Layer: Python / FastAPI

The intelligence service is implemented as a Python `FastAPI` application. Python was the only reasonable choice here: PyTorch, scikit-learn, and the Hugging Face ecosystem are Python-first and would have required awkward bridging in any other language. FastAPI was chosen over Flask or Django for two reasons. Its `async` request handling matches the expected load pattern (individual scoring and planning requests, not bulk data processing), and its automatic OpenAPI documentation makes the contract between the two services immediately visible during development. There is also a practical benefit: FastAPI's request parsing raises clear validation errors when the payload does not match the expected shape, which surfaces interface mismatches early.

5.1.3 Model Training: PyTorch and scikit-learn

Model training uses PyTorch for the neural planner and scikit-learn for the classifier-based scorer. The two frameworks complement each other well. scikit-learn's pipeline abstraction is ideal for the text-based classifier (TF-IDF vectorization followed by a gradient-boosted classifier), while PyTorch handles the more structured multi-head MLP for planning. Both produce artefacts that can be loaded at inference time without framework-specific servers — `.pt` files for PyTorch and `.joblib` files for scikit-learn — which kept the intelligence service's runtime dependencies minimal.

5.1.4 Persistence

Persistence is intentionally lightweight. Audit events are written to an append-only JSONL file (`data/audit.jsonl`), chosen because JSONL requires no schema migration, is human-readable, and can be tailed in real time during a demo. Cases — which represent the ongoing state of an investigated incident — are stored in SQLite via the `SqliteCaseManager`. SQLite was preferred over a full relational database because it requires no server process and runs on any machine, which is appropriate for a single-machine demonstration system.

5.1.5 Configuration

All configuration is driven by environment variables, loaded from a `.env` file at startup by `EnvFileLoader`. This separates code from operational settings — switching from the custom neural planner to the seq2seq planner, for instance, requires only a one-line change to `.env` without recompiling anything. This pattern also makes the system straightforward to reconfigure between different demo scenarios.

5.2 Core Engine Implementation

The orchestration engine is the beating heart of the system. Every alert that enters the pipeline passes through a deterministic sequence of stages, each isolated behind an interface, each recorded in the audit log. The design goal was simple: any single stage could fail loudly without corrupting the rest, and an engineer reading the main processing loop should be able to understand the entire system at a glance.

5.2.1 The Orchestration Pipeline

The central class is `AgentOrchestrator`. It holds references to every stage of the pipeline as interface-typed dependencies, injected at construction time via the standard Microsoft DI container. In tests, any stage can be substituted with a stub. In production, the full implementation is wired up once in `Program.cs` and left alone. The primary processing method, simplified to its essential structure, reads as follows:

```
// Pull a batch of raw alerts from the SIEM connector
RawAlert[] alerts = await connector.PullAlertsAsync(request, ct);

foreach (RawAlert raw in alerts)
{
    // Stage 1: normalize and enrich
    EnrichedAlert enriched = await normalization.ProcessAsync(raw, ct);

    // Stage 2: score the threat
    ThreatAssessment assessment = scorer.Score(enriched);

    // Stage 3: generate a response plan
    DecisionPlan plan = planner.Plan(enriched, assessment, context);
```

```
    // Stage 4: evaluate plan against policy
    PolicyDecision decision = policy.Evaluate(plan, assessment, enriched, context);

    // Stage 5: execute approved actions
    ExecutionReport report = await execution.ExecuteAsync(
        enriched, assessment, plan, decision, execContext, ct);

    // Stage 6: persist audit record and update case
    audit.Log(raw, enriched, assessment, plan, decision, report);
    caseManager.OpenOrUpdate(assessment, plan, decision);
}
```

Each stage boundary is also an exception boundary. If normalization fails, the pipeline records a `NormalizationFailed` audit event and moves on to the next alert rather than terminating. The same principle applies to planning. Scoring goes a step further: a `ScoreSafe` wrapper catches exceptions and returns a low-confidence fallback assessment, because a scoring failure should never block the rest of the pipeline. A degraded score is better than a dropped alert.

This was a deliberate choice over an alternative where failures terminate the run. In a real SOC environment, partial failures are normal: a model might time out on one alert while processing the next one perfectly. Failing the entire batch over a single bad alert would be operationally unacceptable.

5.2.2 Interface-Driven Architecture

Every major component is defined as a C# interface before it is implemented. The full set of core interfaces is:

- `ISiemConnector`: Abstracts alert sources. Implements `ConnectAsync`, `PullAlertsAsync`, `AckAsync`, and `GetHealthAsync`. Both the Wazuh HTTP connector and the in-memory simulator implement this interface, so the orchestrator does not know which one it is talking to.
- `INormalizationPipeline`: Accepts a `RawAlert` and returns an `EnrichedAlert`. Internally chains validation, field mapping, and enrichment, but the orchestrator sees only one method call.

- `IThreatScorer`: Accepts an `EnrichedAlert` and returns a `ThreatAssessment`. In production this is an HTTP client that calls the Python intelligence service; in tests it is a stub that returns predetermined values.
- `IPlanner`: Accepts an `EnrichedAlert`, a `ThreatAssessment`, and a `PlanningContext`, and returns a `DecisionPlan`.
- `IExecutionPipeline`: Routes approved actions to the correct executor and returns an `ExecutionReport`.
- `IAuditLogger`: Accepts structured `AuditEntry` records and persists them.
- `ICaseManager`: Manages the lifecycle of investigated incidents as persistent cases.

This level of interface coverage may look over-engineered for a research prototype, but it serves a real purpose. The system becomes testable without external infrastructure, and swapping any single component does not disturb the others. The policy engine and approval workflow went through several revisions — their callers never had to change.

5.2.3 Domain Model: Alert Lifecycle

An alert passes through a well-defined lifecycle of types, each richer than the last:

```
RawAlert
  → (normalization) → NormalizedAlert
  → (enrichment)    → EnrichedAlert
  → (scoring)       → + ThreatAssessment
  → (planning)      → + DecisionPlan
  → (policy)        → + PolicyDecision
  → (execution)     → + ExecutionReport
```

The `RawAlert` is the unprocessed SIEM payload — it contains whatever fields the source provided, plus a canonical `AlertId`, `SiemName`, and `TimestampUtc`. The `NormalizedAlert` applies a SIEM-specific mapping registry to extract a common set of fields: `AlertType`, `Severity`, and a structured `Entities` object containing fields like `SrcIp`, `Hostname`, and `Username`. The `EnrichedAlert` layers context on top — an `AssetContext` with asset criticality and environment, a `ThreatIntelContext` with reputation scores and matched tags, and an `IdentityContext` with privilege level.

These types are C# records — immutable value objects that are created once and never mutated. Immutability matters here: it means that any stage receiving an alert is

guaranteed to see the same data that was passed into it, with no risk of a side effect in one part of the pipeline silently changing what another part sees.

5.2.4 Normalization and the Multi-SIEM Mapping Registry

One of the more interesting engineering challenges was handling alerts from heterogeneous SIEM sources. A Wazuh alert structures its fields very differently from a Microsoft Sentinel alert, yet both need to produce a `NormalizedAlert` with the same set of fields. The solution is a `MultiSiemMappingRegistry` that reads the `X-Siem-Name` header from incoming webhook requests and delegates to the appropriate SIEM-specific mapper.

Each mapper knows, for instance, that a Wazuh alert stores the agent hostname at `agent.name` and the attacker IP at `data.srcip`, while a Sentinel alert stores the hostname in `entities[].HostName` and the tactic in `tactics[0]`. Alert type is inferred from the rule name and tactic when an explicit type field is not present. A Sentinel alert with tactic `CredentialAccess` and a title containing "brute force" maps to the `BruteForceUser` alert type.

5.2.5 Enrichment Pipeline

After normalization, the enrichment stage consults up to three data providers, each loaded from a JSON file at startup:

- `AssetInventoryEnricher`: Matches `Hostname` or `HostId` against an assets registry. Returns asset criticality (1–5), environment (`prod`/`dev`/`dmz`), and asset category.
- `ThreatIntelEnricher`: Matches `SrcIp` and `FileHash` against a threat intelligence registry. Returns reputation score (0–100) and matched tags such as `tor-exit-node` or `credential-stuffing`.
- `IdentityEnricher`: Matches `Username` against an identity registry. Returns privilege level, department, and user risk score.

The enrichment providers are loaded lazily from disk and consulted in order. The `DefaultEnrichmentMerger` then folds their results into the `AlertContext`, combining fields from multiple providers into a single coherent context block. This context block is what flows into the Python intelligence service as part of the scoring and planning payload.

The validation step, implemented in `BasicAlertValidator`, runs before enrichment. It enforces a small set of invariants: the alert must have a non-empty `AlertId`, a non-empty `SourceSiem`, and at least one actionable entity (a host, user, or IP address). Alerts that fail validation are rejected with a structured error rather than silently dropped. This "fail loudly" approach prevents downstream components from operating on underspecified inputs.

5.2.6 Webhook Mode and Polling Mode

The engine supports two operating modes, selected at startup. In **polling mode**, the engine calls `PullAlertsAsync` in a loop against a Wazuh API connector, sleeping for a configurable interval between cycles. In **webhook mode**, a lightweight HTTP listener (`WebhookAlertListener`) binds to a local port and calls `HandleAlertAsync` for each incoming POST request, passing the decoded JSON payload as a `RawAlert`.

Webhook mode was the one used for the demo. Alerts are sent by an external replay script (`scripts/replay_alerts.py`), which reads pre-recorded alert JSON files and POSTs them to the listener with a configurable delay between alerts. This gives full control over demo pacing while exercising the same code path that real SIEM webhook integrations would use.

5.3 Intelligence Service

The intelligence service is where the cognitive work happens. It exposes two endpoints — `POST /v1/score` and `POST /v1/plan` — and behind each one lives a configurable backend that can be a rule engine, a trained classifier, a neural network, a seq2seq model, an Ollama-hosted LLM, or any combination of these. From the C# engine's perspective, all of these look the same: send JSON, receive JSON.

5.3.1 Service Startup and Backend Initialisation

When FastAPI starts, a single `_build_services()` function is called to instantiate the scorer and planner backends. It reads environment variables (or, if a model registry file is present, the active profile from that file) and constructs the appropriate scorer and planner objects. The reason for doing this at startup rather than lazily per request is that model loading is expensive: a seq2seq model or a neural network takes several seconds to load from disk. Failing at startup is the better operational choice — it is far easier to

diagnose a missing model file from a startup error than from a 500 response on the first real alert.

The construction logic, simplified to its structure, looks like this:

```
# At startup:
if planner_mode == "custom_nn":
    planner = CustomNNPlanner(model_path)
elif planner_mode == "seq2seq":
    planner = Seq2SeqPlanner(model_path, max_new_tokens=384, num_beams=4)
elif planner_mode == "ollama":
    planner = OllamaPlanner(base_url, model_name, timeout=120)
elif planner_mode == "hybrid":
    planner = HybridPlanner(local_planner, remote_planner)
else:
    planner = RulePlanner()   # Always-available fallback

# Similarly for scorer:
if scorer_mode == "classifier":
    scorer = ClassifierScorer(model_path)
elif scorer_mode == "hybrid":
    scorer = HybridScorer(classifier, ollama_scorer, fallback=RuleScorer())
```

A `RulePlanner` and a `RuleScorer` are always available as last-resort fallbacks. If a model fails to load, or if an LLM endpoint is unreachable, the system degrades gracefully to rule-based behaviour rather than refusing to start.

5.3.2 Request Handling and the LRU Cache

Each POST to `/v1/score` extracts the alert payload, resolves the active model profile, and checks a fixed-size LRU cache (default: 256 entries) before calling the scorer. The cache key is a deterministic JSON serialisation of the alert and the profile name. In practice, many demo alerts are identical or near-identical — a replay of the same event with a different timestamp but the same entities — so the cache significantly reduces redundant model inference.

If the scorer returns `None` — meaning the backend could not produce a result, for instance because an LLM timed out — the service falls back to the `RuleScorer` rather than returning an error. The `/v1/score` endpoint always returns a valid assessment, even if

that assessment was produced by a fallback. The caller does not need to handle scoring failures.

```
@app.post("/v1/score")
async def score(payload, request):
    alert = payload.get("alert") or payload
    key   = cache_key(alert, active_profile)

    if cached := cache.get(key):
        return cached

    result = scorer.score(alert)
    if result is None:
        result = RuleScorer().score(alert)  # guaranteed non-None

    cache.set(key, result)
    return result
```

Planning requests follow the same pattern but without caching, because plans depend on the full assessment context which may vary even for identical alert payloads. If planning raises an exception, the endpoint catches it and returns the output of the `RulePlanner` instead.

5.4 Model Registry and Profile-Based Backend Selection

One of the more useful infrastructure components in the intelligence service is the model registry. The registry is a JSON file (`intelligence/models/registry.json`) that defines named *profiles*, each specifying a scorer type and a planner type with their respective configuration. An example profile looks like:

```
{
  "active": "custom-nn",
  "profiles": {
    "custom-nn": {
      "scorer": {
        "type": "classifier",
        "model_path": "intelligence/models/scorer_classifier"
```

```
      },
      "planner": {
        "type": "custom_nn",
        "model_path": "intelligence/models/planner_nn",
        "action_threshold": 0.50
      }
    },
    "seq2seq": {
      "scorer": { "type": "classifier", ... },
      "planner": { "type": "seq2seq", "model_path": "...", "num_beams": 4 }
    },
    "rule-only": {
      "scorer": { "type": "rule" },
      "planner": { "type": "rule" }
    }
  }
}
```

The `ModelRegistry` class reads this file at startup and builds `ModelProfile` objects from each entry. Scorer and planner instances are *lazily* instantiated on the first request for each profile and then cached — loading a 250M-parameter seq2seq model on every request would be unworkable. The active profile can be overridden per request via either the `modelProfile` field in the JSON payload or the `X-Intel-Profile` HTTP header, which makes it straightforward to compare profiles side-by-side without restarting the service.

The reason for a declarative registry of named profiles, rather than simply hardcoding a single model at startup, is that model comparison becomes a configuration operation rather than a code change. During development, switching between the custom NN planner, the seq2seq planner, and the rule baseline required only changing a single environment variable.

5.5 Threat Scoring Component

The scorer's job is to answer a deceptively simple question: given everything we know about this alert, how serious is it, and why? The answer is a `ThreatAssessment` containing a `severity` (0–100), a `confidence` (0–1), a natural-language `hypothesis` string, and a list of structured `evidence` items.

5.5.1 The Classifier Scorer

The primary scorer in the deployed configuration is the `ClassifierScorer`. It wraps a scikit-learn pipeline trained on a prompt-based representation of alerts — a text string encoding the alert type, severity, confidence, entities, and assessment context into a single paragraph. The classifier predicts one of five severity labels: `benign`, `low`, `medium`, `high`, or `critical`. The label is mapped to a numeric severity score via a calibrated class-to-severity table.

One key implementation detail is how raw classifier probability is converted to a meaningful confidence value. In a five-class problem, raw `predict_proba` values tend to underestimate decisiveness: a top class probability of 0.55 beating the next class at 0.15 is, in practice, a very decisive classification, but reporting 0.55 as the confidence would look weak to a downstream consumer. To address this, label-calibrated confidence floors are applied when the top class beats the runner-up by at least a 1.1× margin:

```
# Label-calibrated confidence floors
_LABEL_CONFIDENCE_FLOOR = {
    "critical": 0.88,
    "high":     0.72,
    "medium":   0.52,
    "low":      0.38,
    "benign":   0.22,
}

raw_conf = float(max(predict_proba(prompt)))
decisive  = sorted_proba[0] >= sorted_proba[1] * 1.1
floor     = _LABEL_CONFIDENCE_FLOOR.get(label, 0.5)

if decisive and raw_conf < floor:
    confidence = floor
else:
    confidence = raw_conf
```

This ensures that a decisive classification of "critical" registers a confidence of at least 88%, which more accurately reflects the model's certainty and makes downstream policy thresholds (which are also expressed as confidence levels) behave sensibly.

5.5.2 Enrichment-Aware Confidence Boosting

One of the more nuanced scoring decisions is what to do when external enrichment context strongly corroborates — or escalates — the alert. Consider a brute-force alert from an IP address with a threat intelligence reputation score of 95/100 and a tag of `tor-exit-node`. A raw classifier might score this at 72% confidence because the alert signal is moderately strong. But a security analyst looking at the same alert would feel much more certain: a known TOR exit node attacking a production asset is very unlikely to be a false positive.

The scoring component incorporates this reasoning through a post-classification enrichment boost:

```
# Threat intelligence corroboration
if ti_reputation >= 90:
    confidence = max(confidence, 0.90)  # near-certain TI match
    severity   = max(severity,   85.0)
elif ti_reputation >= 70:
    confidence = max(confidence, 0.82)  # strong TI corroboration
    severity   = max(severity,   75.0)
elif ti_reputation >= 50:
    confidence = max(confidence, 0.70)  # moderate TI signal

# Asset and identity context
if asset_criticality >= 5:
    confidence = max(confidence, 0.85)
    severity   = max(severity,   80.0)
elif asset_criticality >= 4:
    confidence = max(confidence, 0.78)
    severity   = max(severity,   70.0)

if privileged:
    confidence = max(confidence, 0.75)
```

The `max()` operator is important here: the boosts are floors, not absolute assignments. If the classifier already scores at 92% confidence, the TI boost does not change that. The boosts only activate when external context provides stronger evidence than the model's raw output.

5.5.3 Hypothesis and Evidence Generation

The `hypothesis` field in the assessment is a human-readable sentence that explains the alert in terms a SOC analyst would recognise. It is generated deterministically from the alert type, entities, and enrichment context — there is no LLM call involved, just carefully constructed template logic. A brute-force alert against a known privileged account, from an IP flagged as a TOR exit node, generates:

> *"Multiple failed authentication attempts against account 'root' (87 failed attempts) suggest brute-force activity. Origin identified in threat intelligence (tor-exit-node, brute-force, credential-stuffing; reputation 95/100). High likelihood of malicious intent. — Target is a critical production asset; compromised account has elevated privileges."*

This is assembled from three independent pieces: the base activity description (parsed from alert type and entity values), the threat intelligence narrative (constructed when TI reputation ≥ 70), and the enrichment suffix (added when asset criticality or privilege level is notable). The choice to use deterministic templates rather than LLM generation is intentional — the hypothesis is reproducible, debuggable, and can never hallucinate entity values.

The `evidence` list is similarly constructed: a short array of structured key-value strings (e.g., `alert_type=BruteForceUser`, `src_ip=185.220.101.47`, `ti_reputation=95/100`) that document what signals contributed to the score. An analyst reading the evidence list can immediately see why the system scored the alert the way it did.

5.6 Decision Planning Component

Planning is the most intellectually interesting component in the system. Given a scored alert, the planner must decide what to do: which containment strategy to adopt, which specific actions to recommend, in what priority order, and with which entity parameters. The planner must be both precise and safe — recommending the right action with the wrong parameter (for instance, blocking the wrong IP) is worse than recommending nothing.

5.6.1 Framing Planning as Structured Prediction

A key early design decision was to treat planning as *structured prediction* rather than text generation. The planner's output is not a paragraph describing what should happen.

It is a structured object with typed fields: a `Strategy` enum (one of five values), a list of `PlannedAction` objects (each with a typed `ActionType` and a `Parameters` dictionary), and an integer `Priority`. Action parameters (IP addresses, hostnames, usernames) are populated directly from the entities extracted during normalization — the model never generates entity values, it only decides which actions to include.

The reason for this framing comes down to safety. If a model can generate entity values, it can hallucinate them. An LLM asked to produce a blocking action might generate a plausible-looking but incorrect IP address. By restricting the model's output to structural choices (which action types to include, what strategy to adopt) and filling parameters from the normalized alert, the planner is structurally incapable of hallucinating entity values.

5.6.2 The Custom Neural Network Planner

The primary planner in the deployed system is a custom multi-head MLP (`CustomNNPlanner`) trained entirely from scratch on a dataset of 8,644 structured examples. The decision to build a custom neural network rather than fine-tuning a pretrained language model was motivated by three factors.

First, the task structure is highly regular: five alert types map to five strategies and eight possible action types, drawn from a fixed vocabulary. This is not a general-purpose language understanding problem — it is a structured classification problem with strong domain regularities. A pretrained LLM brings 250 million parameters of general world knowledge that are simply not needed here.

Second, inference speed matters for a real-time pipeline. A 250M-parameter seq2seq model takes 800–1200ms per example on CPU; the custom MLP infers in under 5ms. It can also be loaded entirely in RAM on any development machine without a GPU.

Third, the custom MLP is inherently more interpretable. Its 198-dimensional feature vector is fully documented, and each feature has an explicit semantic meaning. The weights of a fine-tuned T5 model, by contrast, are distributed across attention heads and feed-forward layers in ways that resist simple interpretation.

5.6.3 Feature Engineering

The MLP receives a fixed-length feature vector assembled from the alert and its assessment. The full feature vector has 198 dimensions, composed as follows:

- **Alert type one-hot** (5 dims): Encodes the alert type as a one-hot vector over the five recognised types (`PortScanFromIp`, `BruteForceUser`, `MalwareHashOnHost`, `SuspiciousProcessOnHost`, `BenignNoise`).
- **Severity and confidence** (2 dims): Normalised severity ($\in [0, 1]$) and confidence ($\in [0, 1]$) from the assessment.
- **Entity presence flags** (5 dims): Binary flags for the presence of `srcIp`, `hostId`, `username`, `fileHash`, and `processName`. These flags serve a dual purpose: they are features for strategy selection, and they are used to mask infeasible actions during training and inference.
- **Confidence and severity buckets** (8 dims): Discretised bins for confidence (<0.3, 0.3–0.6, 0.6–0.85, ≥ 0.85) and severity (<30, 30–50, 50–70, ≥ 70). Buckets were added because the MLP's linear layers struggle to learn sharp threshold behaviour from continuous values alone.
- **Interaction features** (10 dims): Element-wise products of the alert type one-hot with the normalised severity and confidence respectively. These capture, for example, that a brute-force alert at high confidence is qualitatively different from a port scan at the same confidence level.
- **Security keyword presence** (27 dims): Binary membership flags for 27 domain-relevant keywords extracted from the hypothesis and evidence text (e.g., `malware`, `brute`, `privilege`, `lateral`, `forensic`). These capture semantic signals that are not captured by the alert type alone.
- **SIEM source one-hot** (3 dims): Encodes the data source (CIC-IDS2018, MITRE ATT&CK, Mordor), which correlates with the distribution of alert types and entity patterns.
- **ATT&CK technique presence** (1 dim): Binary flag for whether a MITRE ATT&CK technique ID is present in the raw payload, which indicates higher-confidence attribution.
- **TF-IDF text features** (128 dims): Bigram TF-IDF weights computed over the concatenated hypothesis and evidence text, fitted on the full training corpus. TF-IDF features capture lexical patterns that span multiple keywords and are complementary to the keyword presence flags.

The architecture of the MLP itself follows a three-layer shared encoder with three independent output heads:

```
Input (198 dims)
  → Linear(198, 512) + LayerNorm + GELU + Dropout(0.3)
  → Linear(512, 256) + LayerNorm + GELU + Dropout(0.2)
  → Linear(256, 128) + LayerNorm + GELU
  --> Shared representation (128 dims)
  +-- Strategy head:  Linear(128, 64) + GELU + Linear(64, 5)  --> softmax
  +-- Action head:    Linear(128, 64) + GELU + Linear(64, 8)  --> sigmoid
  \-- Priority head:  Linear(128, 32) + GELU + Linear(32, 1)  --> sigmoid * 100
```

GELU activations were used in preference to ReLU because they produce smoother gradient flow, which was found empirically to reduce training variance. LayerNorm was added at each shared layer because the feature vector mixes dimensions with very different scales (binary flags alongside TF-IDF weights alongside normalised continuous values), and normalisation prevents the larger-magnitude dimensions from dominating the early layers. The total parameter count is approximately 270,000 — roughly three orders of magnitude smaller than a fine-tuned T5 model.

5.6.4 Multi-Head Training with Masked Action Loss

Training uses three loss functions simultaneously:

```
# Per-batch training step
strategy_loss = CrossEntropyLoss(strategy_logits, strategy_targets)
action_loss   = BCEWithLogitsLoss(action_logits, action_targets,
                                  weight=class_weights) * entity_mask
priority_loss  = MSELoss(priority_pred, priority_targets)

total_loss = 1.0 * strategy_loss
           + 0.5 * action_loss.mean()
           + 0.1 * priority_loss
```

The `entity_mask` in the action loss is particularly important. For each training example, action labels that are infeasible given the available entities are masked to zero in the loss computation. For instance, if an example does not have a source IP, the `BlockIp` loss is zeroed out for that example regardless of what the target label says. Without this masking, the model would be penalised for not predicting `BlockIp` on examples where no IP was present, which introduces incoherent gradient signal. The masking ensures

that the model learns the association between action types and entity availability, not just the association between alert types and action labels.

The strategy loss weight is set to 1.0 because strategy selection is the highest-level decision — the action selection should follow from the strategy, not lead it. Action loss is weighted at 0.5 and priority at 0.1, reflecting the fact that priority is the least critical output for policy correctness.

5.6.5 Enrichment Post-Processing Layer

The MLP was trained on a dataset that did not include enrichment context (asset criticality, threat intelligence, identity privilege). Rather than retrain the model with additional features — which would require new training data and a longer feature vector — a deterministic post-processing layer was added that applies rule-based corrections to the model output after inference.

The reasoning here is straightforward. Enrichment context provides reliable, high-priority signals that should always override the model's base output when present. A human analyst would think the same way: if the model says "observe more" for an alert on a criticality-5 production server with a 95/100 TI reputation score and 90% confidence, that recommendation needs to be escalated, regardless of what the model's statistical patterns suggest. The post-processing layer encodes this reasoning explicitly:

```
# Strategy escalation for critical assets
if asset_criticality >= 5 and confidence >= 0.70:
    if strategy in ("ObserveMore", "NotifyOnly"):
        strategy = "Contain"
    elif strategy == "Contain":
        strategy = "ContainAndCollect"

# Privileged identity: don't leave at ObserveMore
if privileged and confidence >= 0.65 and strategy == "ObserveMore":
    strategy = "Contain"

# Force forensic collection for critical assets
if asset_criticality >= 4 and confidence >= 0.65:
    if "CollectForensics" not in current_action_types:
        actions.append({"type": "CollectForensics", ...})
```

```
# Priority floors by asset criticality
floor = {5: 85, 4: 70, 3: 55}.get(asset_criticality, 0)
if floor and priority < floor:
    priority = floor
```

Every override appends a human-readable note to the plan's `rationale` list, which is surfaced in the console output as `[NOTE]` lines. This preserves full traceability: an analyst can always see exactly which enrichment signals caused the model output to be modified.

5.7 Policy and Safety Layer

The policy engine is the safety boundary between intelligence and execution. It receives a fully formed `DecisionPlan` and classifies each action into one of three outcomes: **Approved** (execute autonomously), **Pending** (requires human approval), or **Denied** (blocked unconditionally). This classification is the last chance to intercept a harmful recommendation before it reaches the execution layer.

5.7.1 Three-Tier Decision Model

The policy evaluation proceeds in two passes. The first pass checks for hard denials — conditions where no approval path exists:

```
// Hard denial conditions
if (!catalog.TryGet(action.Type, out var def))
    → Denied: "Unknown or unsupported action type."

if (action is missing required parameters)
    → Denied: "Missing required parameters."

if (action.Type is in ForbiddenActionsByEnvironment[environment])
    → Denied: "Action forbidden in this environment."

if (confidence < MinConfidenceForAutonomy
    && action.Type is not in SafeLowConfidenceActions)
    → Denied: "Confidence below minimum for autonomous action."
```

```
if (criticality >= CriticalAssetThreshold
    && action.Type is in ForbidActionsOnCriticalAssets)
    → Denied: "Action forbidden on critical assets."
```

Actions that survive the denial pass then enter the approval gate check:

```
// Approval gate conditions
if (def.RequiresApprovalByDefault)
    → Pending: "Catalog marks action as approval-required."

if (action.Risk >= RiskApprovalThreshold)
    → Pending: "Risk exceeds approval threshold."

if (environment == "prod" && action.Type is in RequireApprovalInProd)
    → Pending: "Requires approval in production."

if (criticality >= CriticalAssetThreshold
    && action.Type is in RequireApprovalOnCriticalAssets)
    → Pending: "Target asset is critical."
```

Actions that pass both checks are returned as Approved. The two-pass structure — hard denials first, approval gates second — means that a policy rule can always guarantee a denial regardless of any other configuration, while approval gating is additive and can be layered on top.

5.7.2 Entity Presence Validation

One detail that is easy to underestimate is that the policy engine independently validates entity presence for every action, even after the planner has already checked it. This redundancy is intentional. The planner runs in the Python intelligence service and communicates over HTTP; the policy engine runs in the C# orchestration layer. Validating parameters in both layers means that a bug in the planner — or a malformed HTTP response — cannot result in an action being executed with missing parameters.

The parameter validation logic uses alias-aware key lookup: for `BlockIp`, the presence of any of `src_ip`, `srcIp`, `source_ip`, or `ip` in the action parameters is accepted. This robustness against naming variation turned out to matter during integration testing, where different SIEM sources used different field name conventions.

5.7.3 Action Catalog

The `ActionCatalog` is a registry of all recognised action types, each described by an `ActionDefinition` that specifies:

- **RequiredParameters**: The set of parameter keys that must be present for the action to be executable.
- **RequiresApprovalByDefault**: Whether the action enters the approval queue by default, regardless of risk/impact scores.
- **DefaultRisk** and **DefaultImpact**: Baseline scores (0–100) used when the planner does not specify explicit values.

`IsolateHost` and `BlockIp` are configured with `RequiresApprovalByDefault = false` and reasonable default risk scores, meaning they can be auto-executed when confidence and asset criticality conditions are met. `QuarantineFile` and `KillProcess` carry higher default impact scores and are denied on critical assets, reflecting the operational concern that killing a process on a production server may cause service disruption.

5.8 Action Execution and Response

The execution pipeline routes approved actions to the correct executor and records every outcome. Each executor implements the `IActionExecutor` interface, which exposes a single method: `CanHandleAsync(action)` to determine if this executor handles the given action type, and `ExecuteAsync(action, context)` to carry out the action.

In the deployed configuration, the following executors are registered:

- `FirewallExecutor`: Handles `BlockIp` and `UnblockIp`. In dry-run mode, logs the intended action without making any network call.
- `HostIsolationExecutor`: Handles `IsolateHost` and `UnisolateHost`. In dry-run mode, simulates the isolation response.
- `UserAccessExecutor`: Handles `DisableUser` and `EnableUser`.
- `TicketingExecutor`: Handles `OpenTicket`.
- `NotificationExecutor`: Handles `Notify`.

The `ExecutorRouter` dispatches each action to the first registered executor that claims it, then collects results into an `ExecutionReport`. Actions for which no executor claims responsibility are recorded as failed. In dry-run mode (the default for all demo scenarios), every executor simulates its action — writing the intended operation to the audit log without making any real network or system call. The dry-run flag is propagated from the environment configuration to the `ExecutionContext` and respected by every executor.

The dry-run default was a deliberate safety choice. During development, testing, and demonstration, the system should never make real infrastructure changes. An engineer working on the planner or policy engine should not have to worry about accidentally triggering a firewall rule on a real network. The system's first posture is always "recommend and record" — real execution requires an explicit opt-in.

5.9 Audit, Traceability, and Console Output

Auditability was treated as a first-class requirement from the start. Every stage of the pipeline writes a structured entry to the audit log: normalization failure, scoring completion, planning completion, policy decision, and execution outcome all produce entries with a shared `CorrelationId` that links them to the same alert. For any alert, the full decision trace can be reconstructed by filtering the JSONL audit file on its correlation ID.

5.9.1 Demo Trace Writer

In addition to the audit log, the system can optionally write a structured demo trace file (`data/demo_trace.json`) via the `IDemoTraceWriter` interface. Each trace entry is a complete snapshot of the alert's state at the end of processing: the raw alert, the enriched alert, the assessment, the plan, the policy decision, and the execution report. This file is what the demo dashboard reads to reconstruct the full reasoning chain for display in the UI.

5.9.2 Console Output

The console output during a live demo is designed to communicate the system's reasoning at a glance without requiring the viewer to read JSON files. Each processed alert produces a structured block:

```
----------------------------------------------------------------------
```

```
  ALERT  1a2b3c...  [WAZUH]
  Rule : Multiple failed SSH logins (sshd)
  Type : BruteForceUser  |  Severity: 85/100
  Asset: web-server-prod  criticality=5/5  env=prod
  TI   : reputation=95/100  tags=[tor-exit-node, brute-force]
  Ident: userId=root  privileged=True  dept=Infrastructure
  Score: confidence=90%  severity=85
         "Multiple failed auth attempts against 'root' (87 attempts). ..."
  Plan : strategy=ContainAndCollect  priority=85  actions=4
         [NOTE  ] Strategy upgraded to ContainAndCollect -> critical asset.
         [NOTE  ] CollectForensics added -> criticality=5/5.
         [NOTE  ] Priority floored to 85 -> asset criticality=5/5.
         [APPROVED] BlockIp  (185.220.101.47)
         [APPROVED] IsolateHost  (web-server-prod)
         [APPROVED] CollectForensics
         [PENDING ] DisableUser  (root)  -- target identity is privileged.
------------------------------------------------------------------------
```

ANSI colour codes are used throughout: red for high severity and denied actions, yellow for pending actions and moderate severity, green for approved actions and high confidence, cyan for structural elements. Colour output is automatically disabled when the terminal does not support it, so the output degrades cleanly to plain text in CI environments or when piped to a file.

At session end (triggered by Ctrl+C), the `PrintExecutiveSummary()` method produces a high-level statistical summary: total alerts processed, breakdown by SIEM source, strategy distribution, action outcome counts (approved / pending / denied), and the highest-severity alert seen during the session.

5.10 Deployment and System Setup

5.10.1 Single-Command Startup

The system is designed to start from a single command after configuring the `.env` file. The C# engine reads the `.env` file via `EnvFileLoader` at process startup, then launches the Python intelligence service as a managed child process via `IntelligenceProcessManager`. The manager starts the FastAPI service with `uvicorn`, polls the `/health` endpoint until the service responds, and then signals the C# engine to continue with the rest of startup.

The result is that an engineer running the demo does not need to start two terminals or remember to launch the intelligence service separately — starting the C# engine is the only required step.

5.10.2 Webhook Demo Setup

The typical demo setup uses the following sequence:

1. Start the C# engine (webhook mode is selected when `ORCHESTRATOR_WEBHOOK_URL` is set in `.env`).
2. The startup banner is printed, confirming the model profile, enrichment provider count, webhook URL, and dry-run mode.
3. Run the replay script: `python scripts/replay_alerts.py -siem demo -delay 2`.
4. The script sends six production-quality enriched alerts (three Wazuh and three Sentinel) with a two-second delay between each.
5. Each alert is processed through the full pipeline and a formatted summary is printed to the console.
6. Press Ctrl+C to terminate and print the session executive summary.

The `-siem demo` mode was added specifically for the professor demonstration. It selects only the enriched, high-quality alerts from the full dataset, skipping the generic synthetic alerts that have no enrichment context and produce less informative output.

5.10.3 Environment Configuration Reference

All behavioural parameters are controlled by environment variables, which are documented here for completeness:

- `ORCHESTRATOR_WEBHOOK_URL`: If set, starts the engine in webhook mode on the specified URL (e.g., `http://localhost:5050/`).
- `ORCHESTRATOR_DRY_RUN`: If `true` (the default), no real infrastructure actions are taken.
- `INTEL_ACTIVE_PROFILE`: Selects the model registry profile (e.g., `custom-nn`, `seq2seq`, `rule-only`).

- `ENRICHMENT_DATA_PATH`: Path to the directory containing `assets.json`, `threat_intel.json`, and `identities.json`.
- `CASE_DB_ENABLE`: If `true`, enables SQLite case persistence.
- `DEMO_TRACE_PATH`: If set, writes the full demo trace to the specified JSON file.

This environment-driven approach means that the same binary can be reconfigured for different scenarios — local development, professor demo, and (hypothetically) production deployment — by changing a single file, not the code.

Dataset Pipeline and Training

One of the most underestimated parts of building an intelligent security system is everything that happens before any model ever trains: sourcing the right data, normalising it into a consistent representation, deciding what the model should actually learn, and discovering—often the hard way—that the first attempt at all of the above is wrong. This chapter documents that process honestly. The training pipeline went through multiple complete redesigns, from an initial assumption that a pretrained language model would "just work," through fine-tuning experiments with a large security LLM, through seq2seq models, and finally to a purpose-built lightweight neural network that outperforms all of its predecessors on the evaluation benchmark. Each step taught something that the next step needed.

6.1 Security Datasets and Their Roles

No single publicly available dataset covers the full space of alert types, entity patterns, and response scenarios this system needs to handle. The dataset pipeline draws on four distinct sources, each contributing something the others cannot, combined with a synthetic generation layer that fills gaps and enforces coverage balance.

6.1.1 MITRE ATT&CK (STIX)

The MITRE ATT&CK framework [7] provides a structured taxonomy of adversarial tactics, techniques, and procedures. It is distributed in STIX 2.1 format, a machine-readable standard for threat intelligence exchange. For this project, the ATT&CK STIX data serves two purposes. First, it provides authoritative labels: the technique ID attached to a simulated alert can be used to confirm that the alert type classification is grounded in real-world adversary behaviour rather than invented categories. Second, the

technique descriptions provide vocabulary for hypothesis text generation—the language used to describe a lateral movement technique in ATT&CK is the same language a SOC analyst would use, which helps the generated training data sound authentic.

The STIX data is downloaded via a dedicated script and parsed using the `stix2` Python library. Relevant technique metadata—ID, name, tactic mapping, and description—is extracted and stored locally for lookup during alert generation.

6.1.2 Mordor / Security Datasets

The Mordor project, now called Security Datasets [14], provides pre-recorded telemetry from realistic attack emulations run in a controlled lab environment. The datasets contain Sysmon process creation events, registry modifications, network connections, and Windows Event Log entries captured during actual execution of offensive toolkits. This data is valuable because it shows what attacks look like in raw logs—not as abstract technique descriptions, but as concrete process names, command-line arguments, and file paths.

For this project, the Mordor data contributes two things: realistic entity values for the training examples (real hostnames, real process names, real registry paths) and realistic entity *co-occurrence* patterns. A Mimikatz execution on a domain controller, for instance, reliably produces both a process creation event and a network connection. That helps the model learn which entity types to expect together for each attack type. These co-occurrence patterns proved critical for teaching the planner when it makes sense to combine `IsolateHost` with `KillProcess` versus treating them as alternatives.

6.1.3 CIC-IDS2018

The Canadian Institute for Cybersecurity Intrusion Detection System dataset (CIC-IDS2018) [15] is a large labelled network flow dataset containing attacks across seven categories: brute force, heartbleed, botnet, DoS, DDoS, web attacks, and infiltration. The flows are labelled at the packet level, with detailed statistics—packet lengths, inter-arrival times, flag counts—that allow realistic severity calibration.

For the planner training data, CIC-IDS2018 contributes the network-level perspective: port scan alerts naturally associate with source IP entities and `BlockIp` actions; brute force alerts associate with username entities and `DisableUser` actions; the severity distributions by attack category inform the numeric ranges used in synthetic alert generation. The dataset is large (over 16 million flows across 10 days of simulated network traffic), so only the labelled attack flows are used, sampled to avoid class imbalance.

6.1.4 DARPA Transparent Computing (TC)

The DARPA Transparent Computing programme [16] produced provenance-rich datasets capturing multi-stage attack scenarios with full system call traces. Unlike the other datasets, TC data includes temporal dependencies: the provenance graph connects a malicious email attachment to the browser process that opened it, to the child process that spawned, to the registry key it modified, and to the external C2 connection it established. This richer structure informs how the system should reason about alert chains rather than treating each alert as isolated.

For training purposes, the TC data is used to construct malware and suspicious process scenarios with realistic entity chains—alerts that come with a host ID, a process name, and a file hash, which is the complete entity set needed for a `QuarantineFile` response action. Without this dataset, file hash entities were severely under-represented in training, causing early models to consistently miss the `QuarantineFile` action even when the alert clearly indicated malware.

6.1.5 Synthetic Scenario Generation

Despite the above sources, no combination of real datasets provides sufficient coverage of all five alert types, all eight action types, and all five strategy labels in balanced proportions. Real datasets are naturally imbalanced: port scans and brute force events are common; malware file hash detections with a complete entity set (host, process, hash) are rare. A model trained on this imbalance would learn to over-predict the common cases and under-predict the rare but critical ones.

The synthetic generation layer, implemented in `generate_dataset.py`, addresses this by generating structured training examples algorithmically. For each alert type, a generator function produces a plausible alert with randomised entity values, paired with a scored assessment and a labelled response plan. The generators are not naive random—they encode domain knowledge. A brute force generator, for instance, samples a failure count from a realistic distribution (50–500 attempts), assigns a severity in the 60–85 range, and generates a plan that includes `DisableUser` when a username entity is present and `BlockIp` when a source IP is present, falling back to `NotifyOnly` when neither is available:

```
def _gen_brute_force(rng):
    src_ip   = _random_ip(rng)
    username = rng.choice(["admin", "root", "service_acct", ...])
    attempts = rng.randint(50, 500)
```

```
has_ip   = rng.random() > 0.3   # 70% of cases have src IP
has_user = rng.random() > 0.1   # 90% of cases have username

alert = { "type": "BruteForceUser",
          "severity": rng.randint(60, 85),
          "entities": {
              "srcIp":    src_ip if has_ip else None,
              "username": username if has_user else None } }

# Actions conditioned on which entities are actually present
actions = []
if has_ip:   actions.append({"type": "BlockIp",
                             "parameters": {"src_ip": src_ip}})
if has_user: actions.append({"type": "DisableUser",
                             "parameters": {"username": username}})
actions.append({"type": "OpenTicket", "parameters": {}})
```

This entity-conditional generation was not present in the first version of the generator. The early version always generated all entities and always generated all actions, which meant the model could never learn the correct mapping between entity presence and action feasibility. When a `BlockIp` action appeared in every training example regardless of whether a source IP was present, the model learned to predict `BlockIp` unconditionally—and then the policy engine had to reject those actions at runtime because there was no IP to block. This is a concrete example of a data quality problem that looked like a model quality problem until traced back to its source.

6.2 Dataset Processing and Normalisation

Each data source arrives in a different format. STIX is JSON-LD; Mordor is JSON event logs; CIC-IDS2018 is CSV network flows; DARPA TC is compressed provenance graphs. Before any training can happen, all of these must be converted into the same canonical alert schema that the production system uses, so that the model is trained on inputs that are structurally identical to what it will see at inference time.

6.2.1 Schema Mapping

The canonical alert schema used for training mirrors the compact JSON structure that the production `NormalizationPipeline` produces:

```
{
  "type":       "BruteForceUser",
  "severity":   75,
  "sourceSiem": "cic-ids",
  "entities": {
    "srcIp":       "192.168.1.50",
    "username":    "admin",
    "hostname":    null,
    "fileHash":    null,
    "processName": null
  },
  "rawPayload": {
    "attempts":      237,
    "failed_logins": 237,
    "rule_id":       "5710"
  }
}
```

Mapping real data to this schema requires SIEM-specific extractors. A CIC-IDS2018 CSV row with label "Brute Force" maps to the `BruteForceUser` type; the source IP from the "Source IP" column maps to `srcIp`; the packet count maps to the `attempts` raw field. A Mordor Sysmon event with Event ID 1 (process creation) maps to `SuspiciousProcessOnHost`; the `Image` field maps to `processName`; the `Computer` field maps to `hostname`.

One decision that turned out to matter significantly: fields that do not exist in the source data (such as `fileHash` for a network flow event) are explicitly set to `null` rather than omitted. A model trained on examples where absent fields are simply missing rather than explicitly null learns a different—and weaker—representation of entity absence. When `null` is explicit, the model learns "this field is present but empty"; when the field is missing, the model cannot distinguish between "not applicable" and "not yet extracted." Explicit nulls produced noticeably better entity-awareness in downstream planner predictions.

6.2.2 Label Assignment and Severity Calibration

Severity labels for the scorer training data are assigned semi-automatically. For each alert type, a severity range is defined based on domain knowledge and calibrated against the severity distributions in the real datasets:

- **MalwareHashOnHost**: 75–95 (a known-bad file hash is near-definitive evidence)
- **BruteForceUser**: 60–85 (severity scales with failure count and account type)
- **SuspiciousProcessOnHost**: 50–80 (broad range; high false positive rate)
- **PortScanFromIp**: 40–70 (reconnaissance only; rarely immediately harmful)
- **BenignNoise**: 0–20 (must score very low to train the model to suppress false positives)

Within each range, severity is modulated by context: higher failure counts increase brute force severity; a file hash entity matching a known-bad pattern pushes malware severity to the top of its range; alerts from external IP ranges receive higher severity than those from internal subnets.

6.2.3 Action Balancing

A significant problem discovered during early evaluation was that the planner training data was heavily imbalanced in its action labels. `OpenTicket` and `Notify` appeared in nearly every example; containment actions (`BlockIp`, `IsolateHost`, `QuarantineFile`, `KillProcess`) appeared in far fewer. The model learned to default to notification-only responses even for high-severity threats—because the training data had taught it that notification was almost always the right answer.

The fix was a deliberate action-balancing step in the dataset builder. The builder first constructs the full unbalanced dataset, then up-samples examples that contain under-represented containment actions until each action type appears at a minimum frequency. This is different from standard label balancing because one training example can contain multiple actions; the balancing must operate at the action-occurrence level, not the instance level. After balancing, the distribution of containment actions in the final 8,644-example training set reached workable proportions across all eight action types, with containment actions collectively present in over 60% of examples rather than the 20% seen in the unbalanced version.

6.3 Training Data Format Evolution

The format of the training prompt went through three major revisions, each driven by a specific failure mode observed in the model trained on the previous version.

6.3.1 Version 1: Full JSON Prompt

The first format sent the complete alert and assessment as pretty-printed JSON objects:

```
AlertSummary: {
  "type": "BruteForceUser",
  "entities": { "srcIp": "10.0.1.50", "username": "admin", ... },
  ...
}
AssessmentSummary: { "severity": 75, "confidence": 0.80, ... }
Generate a response plan:
```

This format had two problems. First, for seq2seq models, the prompt was 800–1,200 tokens before the response began, leaving little room for the completion within the model's context window. Second, the model had to parse nested JSON to extract entity values for action parameters—and it would reliably hallucinate plausible-looking but incorrect IP addresses rather than copying the exact value from the input.

6.3.2 Version 2: Canonical Flat Format

The second format flattened the alert into a structured but non-JSON representation. This was shorter and clearer, but it still retained a JSON `Entities` block, which required the model to parse JSON-within-text to populate action parameters—still leaving room for hallucination.

6.3.3 Version 3: Compact Pipe-Delimited Format (Final)

The final format, used for the 8,644-example dataset, uses a flat key-value prompt and a pipe-delimited completion:

```
Type: BruteForceUser
Severity: 75.0  Confidence: 0.80  (high)
Entities: {"srcIp": "10.0.1.50", "username": "admin", ...}
Hypothesis: Multiple failed logins from external source.
Evidence: ["src_ip=10.0.1.50", "attempts=237", ...]
Completion: Contain|BlockIp(src_ip=10.0.1.50),DisableUser(username=admin),
            OpenTicket,Notify|priority=80
```

The pipe-delimited completion format was chosen for a specific technical reason that is easy to miss: the flan-t5-base tokeniser (SentencePiece) cannot generate the `{` and `}` characters. This makes it structurally impossible to train a T5 model to output JSON. The pipe-delimited format encodes the same information using only characters that SentencePiece can produce. At inference time, a dedicated parser (`parse_pipe_completion`) reconstructs the full structured plan from the pipe-delimited string, then `sanitize_plan` validates it before it is returned to the C# engine.

This is a good example of a constraint imposed not by the problem but by a tooling choice—a pattern that appeared repeatedly in the training pipeline. Discovering it required actually trying to train on JSON-format completions and noticing that the model's outputs never contained curly braces, then tracing that back to the tokeniser's vocabulary.

6.4 The Scorer: From Non-Functional Baseline to Calibrated Classifier

The scorer converts a normalised alert into a numerical severity score, a confidence value, a hypothesis, and an evidence list. Getting this right took three distinct attempts, each revealing a different class of failure.

6.4.1 Attempt 1: Zero-Shot LLM (baron-security)

The first attempt assumed that a security-focused language model, prompted with the alert payload, would produce reasonable risk scores without any task-specific training. The model was `baron-security`, an Ollama-hosted LLM fine-tuned on security content.

The results were unambiguous. Every alert—port scans, brute force attacks, confirmed malware hash detections, and benign DNS queries—received the exact same response:

severity: 0, confidence: 0, hypothesis: "No malicious activity detected."

Average severity across all six test alerts was 0.0; average confidence was 0.0. No evidence was provided for any alert. The model's instinct toward caution and non-committal safety, which is appropriate for many LLM applications, made it completely useless as a threat scorer. It had learned that the safe answer is always to deny any threat, and nothing in its zero-shot prompt could override that.

This failure was surprising because the assumption had been that a *security-focused* LLM would at minimum recognise that a confirmed malware hash deserved a non-zero severity. The reality was that no amount of prompt engineering fixed it—the model's learned behaviour was too strong. Fine-tuning was required.

6.4.2 Attempt 2: QLoRA Fine-Tuning on Foundation-Sec-8B

The second attempt fine-tuned Foundation-Sec-8B [17], an 8B-parameter security-focused model, using QLoRA [18] (4-bit NF4 quantisation with LoRA [19] rank 16) on a dataset of 3,270 synthetic prompt-completion pairs:

```
lora_config = LoraConfig(
    r=16, lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1, task_type="CAUSAL_LM"
)
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)
```

Training ran for 2 epochs on an RTX 5070 Ti (16GB VRAM), taking approximately 1 hour 45 minutes. Final training loss was 0.41. The results were dramatically better than the baseline. Malware alerts now scored severity 90, confidence 0.85; port scans scored 40, confidence 0.55; benign noise correctly scored 0, confidence 0.20. The model produced specific, meaningful hypotheses and evidence lists. Threat discrimination was now present and qualitatively correct.

However, the LLM approach carried a significant operational cost. Even with 4-bit quantisation, inference took 2–4 seconds per alert. Exporting the fine-tuned LoRA adapter to an Ollama-compatible format added an additional operational step. And the model's behaviour on out-of-distribution alert patterns was difficult to predict or debug, because the relevant decision logic was distributed across billions of weights rather than expressed in inspectable code.

6.4.3 Attempt 3: TF-IDF Classifier Pipeline (Final)

The third approach recognised that scoring is fundamentally a classification problem. Given the alert type and context, the severity label (benign / low / medium / high

/ critical) can be predicted by a discriminative classifier trained on the same prompt-completion pairs. The scorer is a scikit-learn [20] pipeline:

```
pipeline = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2),
                              max_features=10000,
                              sublinear_tf=True)),
    ("clf",   LogisticRegression(max_iter=500,
                                 class_weight="balanced", C=1.0))
])
pipeline.fit(train_prompts, train_labels)
joblib.dump(pipeline, output_path / "model.joblib")
```

Training takes under 30 seconds on CPU; the saved `.joblib` artefact is 2.3MB; inference is sub-millisecond. The classifier achieves 75% severity class accuracy on the held-out evaluation set and a severity MAE of 8.30 points on a 0–100 scale. These numbers appear modest, but they are appropriate for the task: the classifier's role is to produce a severity label that drives confidence calibration and enrichment boosts, not to be the sole determinant of the final score. The TI reputation and asset criticality enrichment boosts modulate the label-derived score significantly for the alerts that matter most.

A key refinement was the **label-calibrated confidence floor**. In a 5-class classification problem, raw `predict_proba` values underestimate decisiveness: a top-class probability of 0.55 beating the runner-up at 0.15 is an extremely decisive prediction relative to the 0.20 random baseline, yet reporting 0.55 as the confidence would cause the policy engine to treat the assessment as uncertain. The calibrated floors apply label-specific minimum confidence values when the classifier makes a decisive prediction (top class $\geq$ 1.1$\times$ runner-up). A decisive prediction of "critical" registers at least 88% confidence; a decisive "high" registers at least 72%. This makes the downstream confidence thresholds in the policy engine behave as intended.

6.5 The Planner: Four Approaches to Structured Prediction

The planner's training journey was more complex than the scorer's, because planning is a harder problem: the output is not a single label but a structured object with multiple interdependent fields that must be internally consistent and entity-grounded.

6.5.1 Attempt 1: Zero-Shot LLM

The same model that failed on scoring also failed on planning, though differently. Rather than returning empty outputs, it defaulted to the same passive response for every alert: `strategy: NotifyOnly`, `actions: [OpenTicket, Notify]`, `priority: 50`. Every alert—benign DNS query and confirmed malware execution alike—received the same treatment. Containment actions were never suggested. The response was also wrapped in a nested `"plan": {...}` key that the C# engine's JSON deserializer did not expect, causing runtime failures.

6.5.2 Attempt 2: Foundation-Sec-8B QLoRA Fine-Tuning

Fine-tuning Foundation-Sec-8B on 3,270 planning examples (2 epochs, 2h 40m, final loss 0.24) produced qualitatively correct results: malware alerts generated `ContainAndCollect` strategies with `QuarantineFile` and `CollectForensics` actions; port scans generated `NotifyOnly` with `OpenTicket`. The schema nesting issue was resolved.

However, new failure modes emerged. Two of the six test alerts produced empty plan objects—the model appeared to fall back to an internal "insufficient confidence" shortcut that bypassed plan generation. More seriously, the model hallucinated entity values in action parameters. It would generate a plausible-looking but incorrect IP address for a `BlockIp` action rather than copying the exact `srcIp` value from the input. For an autonomous response system, this is not a minor accuracy problem—it is a safety failure. Blocking the wrong IP because the model invented a plausible-looking address could cause service disruption on a legitimate host.

This failure motivated the architectural insight described in Chapter 5: action parameters must be populated from the extracted alert entities, not generated by the model. The model's job is to decide *which* action types to include; the entity lookup is deterministic and happens after model inference.

6.5.3 Attempt 3: Seq2Seq Fine-Tuning (flan-t5)

The seq2seq approach reformulated planning as text-to-text translation. The input is the compact alert prompt; the output is the pipe-delimited completion string. Because entity values in the output must match values present in the input, and the model is explicitly trained to copy them rather than generate them, hallucination is substantially reduced.

flan-t5-base [13, 21] (250M parameters) was chosen over decoder-only models for a deliberate reason: its encoder-decoder architecture processes the full input before generating any output, which encourages the model to attend to all entity values before committing to a plan. A causal decoder-only model generates left-to-right from the first token, requiring it to produce action parameters before it has fully processed the entity context in the prompt.

The first training run immediately surfaced a hardware-specific compatibility issue: training with `fp16` mixed precision on an RTX 5070 Ti (Ada Lovelace architecture) produced `NaN` loss from the first training step. Switching to `bfloat16` resolved the issue. The error output only said that the loss was `NaN`—not why. Isolating the root cause required checking whether the issue was in the model, the data, the loss function, or the hardware-precision combination, and systematically ruling out each:

```
# fp16 causes NaN loss on RTX 5070 Ti / Ada Lovelace
# bf16 is required:
training_args = Seq2SeqTrainingArguments(
    bf16=True,
    fp16=False,  # NaN loss if True on Ada Lovelace
    ...
)
```

Multiple dataset variants and training configurations were explored systematically:

- **Canonical dataset, 2 epochs**: strategy accuracy 80%, action F1 77.1% (30 samples). The full JSON prompt format was too long; the model's effective attention was diluted.
- **Compact dataset, 2 epochs**: strategy accuracy 90%, action F1 91.1% (100 samples). The compact prompt format showed substantial improvement.
- **flan-t5-large, compact**: strategy accuracy 80%, action F1 77.1% (30 samples). Identical to flan-t5-base on the same data—the larger model brought no benefit.
- **Compact dataset, 8 epochs**: **strategy accuracy 96.5%, action F1 97.7%** (200 samples). Additional training epochs produced significant gains; the compact format with extended training became the benchmark.

The flan-t5-large result deserves comment. Doubling the parameter count from 250M to 770M produced no improvement over the base model on the same data. This is a

meaningful finding: it suggests that the performance ceiling at this stage was set by data quality, not model capacity. A larger model with the same training data learns the same patterns as a smaller model, just more redundantly. Improving the data—adding TF-IDF features, fixing entity masks, balancing actions—improved results far more than increasing model size.

6.5.4 Attempt 4: Custom Multi-Head MLP (Final)

The custom MLP planner is described architecturally in Chapter 5. Here the focus is on the training process and what made it work.

The model is implemented in PyTorch [22]. The primary training innovation was **masked action loss**. Earlier training datasets implicitly assumed that every action label in the target was feasible for the given input. Many examples included `BlockIp` in the target even when no `srcIp` entity was present, and `QuarantineFile` even when no `fileHash` was available. The BCE loss for these infeasible actions was computed anyway, providing incoherent gradient signal. The fix computes an entity mask for each training example and zeros out the loss for infeasible action slots:

```
# Entity-aware mask: True where the action is feasible given entities
def _build_entity_mask(entities):
    has_ip   = bool(entities.get("srcIp"))
    has_host = bool(entities.get("hostname") or entities.get("hostId"))
    has_user = bool(entities.get("username"))
    has_hash = bool(entities.get("fileHash"))
    has_proc = bool(entities.get("processName"))
    return torch.tensor([
        has_ip,                 # BlockIp
        has_host,               # IsolateHost
        has_user,               # DisableUser
        has_host and has_proc,  # KillProcess
        has_host and has_hash,  # QuarantineFile
        True,                   # CollectForensics (always feasible)
        True,                   # OpenTicket
        True,                   # Notify
    ], dtype=torch.float32)

# Applied in the training loop:
action_loss = BCEWithLogitsLoss(reduction="none")(action_logits, targets)
```

```
action_loss = (action_loss * entity_mask).mean()  # zero out infeasible
```

A second important fix was **label sanitisation**. The training dataset had been built from LLM-generated examples, some of which included rollback actions (`UnisolateHost`, `UnblockIp`, `EnableUser`) in the primary action list rather than the rollback list. These rollback actions are deterministically derived from the primary containment actions and should never be predicted as primary plan actions. Sanitisation strips them from the training targets, ensuring the model never learns to predict them in the wrong position.

TF-IDF bigram features provided the largest single accuracy improvement for the MLP. The initial 70-dimensional structured feature vector captured alert type, severity, confidence, entity flags, buckets, and keyword presence. Adding 128-dimensional TF-IDF features computed over the hypothesis and evidence text increased the feature vector to 198 dimensions. TF-IDF bigrams capture lexical patterns across pairs of adjacent words—the bigram "lateral movement" signals containment more strongly than either word alone. This expansion produced a jump from approximately 88.5% to 96.4% strategy accuracy on the pipeline evaluation.

6.6 Evaluation and Results Comparison

Table 6.1 summarises the planner evaluation results across all approaches. Evaluation uses two metrics: **strategy accuracy** (exact match on the predicted strategy label against the ground-truth label) and **action F1** (macro-averaged F1 on the predicted action type set, with rollback actions stripped from both predictions and targets before comparison). The pipeline evaluation runs the predicted plan through `sanitize_plan` before scoring, ensuring that the metrics reflect post-sanitisation outputs rather than raw model outputs.

The custom MLP matches the 8-epoch seq2seq model within rounding error on a much larger evaluation set (865 samples versus 200), using 270,000 parameters instead of 250 million, and inferring in under 5 milliseconds instead of 800–1,200 milliseconds. The relaxed strategy accuracy metric (where semantically adjacent strategies such as `Contain` and `ContainAndCollect` count as correct for the stricter target) is 96.8% for the MLP—meaning the overwhelming majority of remaining errors are minor escalation differences rather than fundamental strategy mistakes.

The most important finding from this progression is that the task is highly learnable from structured features alone. Once the prompt format was corrected, the dataset was balanced, entity masking was applied, and TF-IDF features were added, all models

TABLE 6.1: Planner evaluation results across all training approaches.

Model	Type	S.Acc.	Act.F1	N	Notes
baron-security (baseline)	Ollama LLM	—	—	6	Passive only; no discrimination
Foundation-Sec-8B (fine-tuned)	QLoRA LLM	0.80	0.84	30	Hallucinated entity values
flan-t5-base (canonical, 2ep)	Seq2Seq	0.80	0.77	30	Full JSON prompt too long
flan-t5-base (compact, 2ep)	Seq2Seq	0.90	0.91	100	Compact format gain
flan-t5-large (action-balanced)	Seq2Seq	0.80	0.77	30	No gain over base model
flan-t5-base (compact, 8ep)	Seq2Seq	**0.965**	**0.977**	200	Best seq2seq result
Custom MLP (pipeline eval)	MLP	**0.964**	**0.972**	865	270K params, <5ms inference

trained on the same data converged to similar performance. The gap between a 250M-parameter pretrained seq2seq model and a 270K-parameter custom MLP is less than one percentage point on both metrics. Performance is bounded by dataset quality, not model capacity. The next round of improvements should invest in richer training data—more diverse entity patterns, more SIEM sources, more edge cases—rather than larger architectures.

6.7 Integration with the Intelligence Service

Once trained, a model must integrate into the live intelligence service without disrupting existing inference. The model registry described in Chapter 5 is the integration point: adding a new trained model requires only a new profile entry in `registry.json` pointing to the saved artefact directory. No code changes are needed in the service itself.

Each artefact is self-describing. The custom MLP saves three files: `model.pt` (PyTorch state dict), `meta.json` (input dimension, per-action probability thresholds, training date, and dataset version), and `tfidf.pkl` (the fitted TF-IDF vectoriser). The metadata file ensures that the inference code loads a model compatible with its own feature extraction logic: if the feature vector dimension changed between training runs, the mismatch is caught at load time rather than at inference time, where a silent wrong answer would be produced. The seq2seq artefact saves as a Hugging Face checkpoint directory loaded directly by `Seq2SeqPlanner`. The classifier scorer saves `model.joblib` and a `meta.json` with the class-to-severity mapping.

All artefact formats are versioned through `meta.json`, making it straightforward to retain multiple model generations and switch between them via the registry without rebuilding anything. During the evaluation phase, this made it easy to run the same alert batch

through the seq2seq planner and the custom MLP planner back-to-back and compare their outputs directly, which is how the data in Table 6.1 was collected.

Evaluation

This chapter covers the quantitative evaluation of the two machine-learning components—the threat scorer and the response planner—and walks through representative incident scenarios end-to-end to verify how the pipeline behaves in practice. The policy engine is evaluated separately for deterministic correctness rather than statistical accuracy, since it contains no probabilistic component.

7.1 Evaluation Setup

7.1.1 Dataset and Split

All evaluation is performed on the synthetic dataset described in Chapter 6. The full dataset contains 8,644 labelled examples generated from five alert types (MalwareHashOnHost, BruteForceUser, SuspiciousProcessOnHost, PortScanFromIp, BenignNoise) using the entity-conditional synthetic generation pipeline. The dataset is split 90/10 into training (7,779 examples) and held-out test (865 examples) sets using a fixed random seed (42) for reproducibility. Stratified sampling ensures each alert type contributes proportionally to both partitions.

7.1.2 Evaluation Metrics

Scorer evaluation uses two metrics. **Severity class accuracy** measures the proportion of test examples where the predicted severity class—one of five labels: benign, low, medium, high, or critical—matches the ground-truth label. **Severity mean absolute error (MAE)** measures the mean absolute difference between the predicted numeric severity and the ground-truth value on the 0–100 scale.

Planner evaluation uses two metrics. **Strategy accuracy** is the exact-match proportion of test examples where the predicted strategy label (ContainAndCollect, Contain, Investigate, NotifyOnly, or Dismiss) matches the ground-truth label. **Action F1** is the macro-averaged F1 score computed over the predicted action type set against the ground-truth set. Before computing either metric, rollback actions (`UnisolateHost`, `UnblockIp`, `EnableUser`) are stripped from both predictions and targets, and each predicted plan is passed through `sanitize_plan` to match the post-sanitisation output a live deployment would produce. This pipeline evaluation protocol avoids inflating scores by crediting raw model outputs that would be silently corrected at runtime.

A **relaxed strategy accuracy** variant additionally counts semantically adjacent strategy pairs as correct—predicting `ContainAndCollect` when the ground truth is `Contain` is treated as a near-miss rather than an error, since both involve active containment and differ only in whether forensic collection is triggered. This metric is reported alongside the exact-match figure to characterise the nature of remaining errors.

7.1.3 Evaluation Environment

Training and evaluation were conducted on a single workstation with an Intel Core i9 processor, 64 GB RAM, and an NVIDIA RTX 5070 Ti GPU (16 GB VRAM). The intelligence service runs as a local FastAPI process. The C# orchestration engine runs as a .NET 9 console application. Inference latency measurements are single-request wall-clock times recorded by the C# engine, including JSON serialisation and the HTTP round-trip to the local intelligence service.

7.2 Detection Performance

7.2.1 Scorer Results

The threat scorer is a TF-IDF logistic regression pipeline trained on 7,779 examples and evaluated on the 865-example held-out test set. It achieves **75.0% severity class accuracy** and a **severity MAE of 8.30** on the 0–100 scale.

The accuracy figure makes most sense relative to the baseline. With five severity classes, random assignment gives 20% accuracy; the classifier hits 75%, a 55-percentage-point margin above chance. The 8.30-point MAE means the average numeric severity deviates from the ground-truth value by less than one severity tier width, since each tier spans roughly 20 points on the 0–100 scale. In practice, the critical errors—predicting benign

for a confirmed malware hash, or critical for a routine DNS lookup—are rare because the discriminative boundary between the extreme classes is well-separated in TF-IDF feature space.

Inference is sub-millisecond on CPU, since the classifier is a sparse matrix multiplication followed by a softmax. The saved `.joblib` artefact is 2.3 MB, making it practical to load on startup and update by retraining without service interruption.

7.2.2 Calibrated Confidence Behaviour

The label-calibrated confidence floor described in Chapter 6 ensures that decisive high-severity predictions register at or above 72% confidence. A decisive prediction is one where the top-class probability exceeds 1.1 times the runner-up. In practice, this means a malware hash alert with a decisive "critical" prediction reliably crosses the 80% confidence threshold required for autonomous containment actions in the policy engine. Without this calibration, raw `predict_proba` values for a five-class problem underestimate decisiveness and would cause the policy engine to treat clear-cut predictions as uncertain, pushing genuine high-severity alerts into the human-approval queue unnecessarily.

7.2.3 Comparison with LLM Baselines

The zero-shot LLM baseline (baron-security) achieved 0% discrimination across all five alert types, assigning zero severity and zero confidence to every alert regardless of content. Prompt engineering could not recover this—the model's bias toward non-committal safety responses dominated any task-specific instruction. The Foundation-Sec-8B QLoRA fine-tune produced qualitatively correct severity ordering—malware scored higher than brute force, which scored higher than benign noise—and was therefore a functional scorer. However, its 2–4 second inference latency per alert made it impractical for near-real-time triage, and its inconsistent output format made automated metric computation on the full test set unreliable. The TF-IDF classifier provides comparable discrimination at sub-millisecond latency with a fully automated, reproducible evaluation pipeline.

7.3 Response Planning Evaluation

7.3.1 Summary of Model Progression

The full progression of planner results across all training approaches is reported in Table 6.1 in Chapter 6. The key result is that the custom MLP achieves **96.4% strategy accuracy** and **97.2% action F1** in pipeline evaluation on the 865-example test set, with a relaxed strategy accuracy of **96.8%**. The 8-epoch flan-t5-base seq2seq model achieves 96.5% strategy accuracy and 97.7% action F1 on a 200-sample evaluation—less than one percentage point apart on both metrics, despite a 900-fold difference in parameter count (250 million versus 270 thousand) and a 160-fold difference in inference latency (800–1,200 milliseconds versus under 5 milliseconds).

7.3.2 Interpretation of Remaining Errors

The 96.8% relaxed strategy accuracy means the majority of the 3.6% of strategy errors are minor escalation differences rather than fundamental misclassifications. A plan that predicts `Contain` when the ground truth is `ContainAndCollect` will block the threat but miss the forensic collection step; an analyst reviewing the audit log can identify and supplement the missing step. By contrast, a plan that predicts `Dismiss` when the ground truth is `ContainAndCollect` would suppress a genuine high-severity threat entirely. The small gap between the exact-match and relaxed accuracy figures (96.4% versus 96.8%) indicates that catastrophic misclassifications of this second kind are rare.

7.3.3 Policy Engine Correctness

The policy engine is a deterministic rule evaluator and is not evaluated with statistical metrics. Its correctness holds by construction: the engine applies entity-presence checks, severity thresholds, irreversibility constraints, and global mode flags independently of the model output. A hard denial is reached before any approval gate, so there is no code path by which a model error or adversarial input can cause a hard-denied action to execute. This structural guarantee—that safety enforcement is independent of model quality—is the key property that distinguishes the system from playbook-based SOAR platforms, where safety depends on the correctness of the playbook author [3].

7.4 Example Incident Scenarios

The following scenarios trace individual alerts through the complete pipeline to illustrate how the scorer, enrichment layer, planner, and policy engine interact to produce a final decision. Each scenario represents a distinct alert type and demonstrates a qualitatively different pipeline outcome.

7.4.1 Scenario 1: Malware Hash Detection — Autonomous Containment

An alert arrives from a simulated EDR connector reporting a file hash match against a known-bad indicator on a production domain controller.

```
Type:            MalwareHashOnHost
Severity:        88    Confidence: 0.91  (critical class, decisive)
HostId:          dc01.corp.local
FileHash:        4a8b9f...e29f
ProcessName:     svchost_spoofed.exe
AssetCriticality: 5 (maximum)    Exposure: internal
```

The enrichment layer matches the file hash against the local threat intelligence store and returns a `malicious` reputation, applying a 1.5 multiplier to the base score. The final enriched score is 94. The planner selects strategy `ContainAndCollect` with actions `IsolateHost`, `KillProcess`, `QuarantineFile`, and `CollectForensics`. The policy engine evaluates each action against three criteria: entity presence (all four pass—host, process, and file hash are available), severity threshold (94 exceeds the isolation threshold of 85), and global mode (containment enabled). All four actions are approved for autonomous execution. Entity parameters (`hostId`, `processName`, `fileHash`) are resolved directly from the enriched alert context, not generated by the model. The audit log records each approval with the specific policy rule satisfied and the entity value used.

7.4.2 Scenario 2: Brute Force Attempt — Conditional Containment

A brute force alert arrives with a source IP but no username entity, indicating a network-level password spray rather than a targeted account attack.

```
Type:            BruteForceUser
```

```
Severity:        72    Confidence: 0.79  (high class)
SrcIp:           203.0.113.47  (external)
Username:        null
AssetCriticality: 3    Exposure: external
```

No threat intelligence hit is returned for the source IP. The external exposure multiplier (1.2) raises the final score to 78. The planner selects strategy `Contain` with actions `BlockIp` and `OpenTicket`. Notably, `DisableUser` does not appear in the plan: because no username entity is present, the entity mask applied during training taught the model to omit this action when the entity is absent, and the policy engine would independently reject it as an entity-presence violation. The policy engine approves `BlockIp` (entity present, severity 78 above the block threshold of 70, action type enabled) and `OpenTicket` (always approved). An analyst receives a medium-priority ticket with the enrichment context and hypothesis.

7.4.3 Scenario 3: Suspicious Process — Investigative Hold

An alert arrives for a process creation event on an internal workstation with a process name pattern associated with living-off-the-land techniques [7].

```
Type:            SuspiciousProcessOnHost
Severity:        61    Confidence: 0.72  (medium class)
HostId:          ws42.corp.local
ProcessName:     certutil.exe  (LOLBin pattern)
AssetCriticality: 2    Exposure: internal
```

Enrichment returns no TI hit for the host; no exposure boost applies (internal). Final score: 61. The planner selects strategy `Investigate` with actions `CollectForensics` and `OpenTicket`. `IsolateHost` is not in the plan, so there is nothing to deny. `CollectForensics` is approved (always feasible; does not affect host availability). The analyst receives a ticket flagging the LOLBin pattern with the process name and host identity attached, allowing targeted investigation without service disruption.

7.4.4 Scenario 4: Benign Noise — Autonomous Dismissal

A routine internal DNS lookup triggers a miscalibrated correlation rule.

```
Type:             BenignNoise
Severity:         6     Confidence: 0.94  (benign class, decisive)
SrcIp:            10.0.1.10  (internal)
AssetCriticality: 1
```

Final score: 6. The planner selects strategy `Dismiss` with no actions beyond an internal audit record. The policy engine approves the null action set. No ticket is opened and no notification is sent. The event is written to the audit log with the dismissal rationale: severity below the notification threshold, benign class predicted with high confidence. This scenario demonstrates the alert suppression capability that directly addresses analyst alert fatigue [2]: routine events are handled entirely without analyst involvement, while the audit record ensures dismissed events remain visible for retrospective analysis.

7.5 Discussion of Results

The evaluation results support four main conclusions.

Performance is bounded by data quality, not model capacity. The near-identical results of the 270K-parameter custom MLP and the 250M-parameter seq2seq model confirm that the response planning task, given the structured feature representation used here, does not require large pretrained language models. Every significant accuracy gain in the training progression came from a data improvement—fixing entity masking, balancing action labels, switching to the compact prompt format, adding TF-IDF bigram features—not from a change in model architecture or size. The practical consequence is that the training and inference cost of the custom MLP is negligible relative to the seq2seq baseline, and both deliver equivalent planning quality.

Structured prediction eliminates the hallucination failure mode. The Foundation-Sec-8B fine-tuning experiment showed that generative planning produces entity hallucination: the model generates plausible but incorrect IP addresses for `BlockIp` parameters rather than copying the exact `srcIp` value from the alert. For an autonomous response system, this is a safety failure, not just an accuracy issue. The structured decomposition used by the custom MLP—classify which actions to include, then resolve entity parameters deterministically from the alert context—eliminates this failure mode entirely. Every parameter in every executed plan can be traced to a field in the original alert.

The policy engine provides a hard correctness floor independent of model quality. A 96.4% strategy accuracy means roughly 3.6% of plans have the wrong strategy. The policy engine does not correct this—it cannot infer the correct strategy from policy rules alone—but it guarantees that the consequences of any model error are bounded.

An incorrectly planned action that violates a hard constraint is blocked before execution. No model error can cause an unsafe action to reach the environment. This separation of learned inference from deterministic safety enforcement is the architectural property that makes the system appropriate for deployment in environments where safety cannot depend on model accuracy alone [5].

Synthetic evaluation establishes a performance baseline, not a deployment guarantee. The 96.4% accuracy figure is computed on test examples from the same generator as the training data. The generator was designed to produce realistic patterns informed by real datasets (CIC-IDS2018 [15], Mordor [14], DARPA TC [16]), but a model evaluated on synthetic data from its own generator faces a fundamentally different distribution than one operating in a live SOC. As Sommer and Paxson observe [10], distribution shift and adversarial adaptation are structural challenges that benchmark evaluations on fixed datasets cannot capture. The results here are best read as a lower bound on planning quality under controlled conditions; real-world validation against genuine incident data is the necessary next step.

Discussion

This chapter puts the evaluation results in broader context, explains why certain design decisions were made, and is honest about the limitations and threats to validity that constrain what the results actually show.

8.1 Advantages of the Proposed System

8.1.1 Explainability as a Structural Property

Most threat scoring systems treat explainability as something added after the fact—a model is trained for accuracy, and then an explanation mechanism like SHAP values or attention weights gets layered on top [10]. The problem is that the explanation is a reconstruction of the decision, not the decision itself.

This system is different. A `ThreatAssessment` cannot be created without a hypothesis string and an evidence list. A `DecisionPlan` requires per-action rationale fields. A `PolicyDecision` records the specific rule that approved, held, or denied each action. These fields are populated at decision time and written verbatim to the audit log—no post-hoc reconstruction. In environments where incident response documentation requires contemporaneous records of automated decisions [5], this design satisfies that requirement structurally.

8.1.2 Safety Guarantees Independent of Model Correctness

The policy engine sits between the learned planner and the execution layer as a deterministic gate, not a recommendation filter. Hard denials are evaluated before any approval gate. There is no code path by which a model error, adversarial input, or distribution-shifted alert can cause a hard-denied action to execute. This is a stronger guarantee

than playbook-based SOAR platforms provide, where safety depends on whoever authored the playbook—or than LLM-based planners, where safety depends on the model not hallucinating unsafe parameter values [3].

The entity-presence check ensures actions requiring a host identifier cannot execute when none is available. These are architectural constraints, not model properties.

8.1.3 Inference Speed and Operational Feasibility

The custom MLP planner infers in under 5 milliseconds end-to-end, including the HTTP round-trip to the Python intelligence service. The TF-IDF scorer runs in sub-millisecond time on CPU. Together, the intelligence layer adds less than 10 milliseconds to the alert processing pipeline.

For comparison, the seq2seq model required 800–1,200 milliseconds per inference. That places it outside the practical range for real-time triage in a high-volume environment. The custom MLP achieves the same planning accuracy at 160 times lower latency. This is not a compromise—structured prediction turns out to be the better design for this task, not just the faster one.

8.2 Comparison with Traditional SIEM and SOAR Approaches

SIEM platforms collect and correlate logs but do not reason about the resulting alerts, assess severity relative to asset context, or take any action [1, 3]. This system starts where the SIEM ends—accepting a normalised alert and adding learned threat assessment, strategy-level planning, policy-gated execution, and an audit trail linking each action to its evidence.

Commercial SOAR platforms execute workflows defined in advance [5]. For well-understood incident types this works. It breaks down on novel patterns, because there is no mechanism to reason about an alert the platform has never seen. This system replaces the static playbook with a trained model that infers the appropriate response strategy from observed alert context. The model is not perfect (3.6% of plans have the wrong strategy), but it generalises from training examples rather than failing silently when nothing matches.

The Foundation-Sec-8B fine-tuning experiment confirmed the core reliability problem with generative LLM planners: the model hallucinates entity values, producing plans that reference IP addresses and hostnames that were never in the alert. For a system

that executes actions against real infrastructure, a plan targeting the wrong resource is a safety failure. The structured prediction approach avoids this by separating *what to do* (learned by the model) from *what to target* (resolved deterministically from the alert after inference).

8.3 Limitations

8.3.1 Evaluation on Synthetic Data Only

All quantitative results are computed on synthetic data generated by the same pipeline that produced the training data. Real SOC environments produce alert distributions that differ in ways a static generator cannot model—rare simulated types may be common in practice, entity patterns the generator treats as independent may be correlated in real deployments, and attacker behaviour evolves over time [10]. The 96.4% strategy accuracy reflects performance under synthetic conditions. Real-world accuracy is unknown and likely lower.

8.3.2 Limited Alert Type Coverage

The system is trained and evaluated on five alert types. Commercial SIEM platforms handle hundreds across network, endpoint, identity, cloud, and application layers. The architecture supports expansion without changes to the orchestration or policy layers, but the training and validation work for production-grade coverage is substantial.

8.3.3 Simulated SIEM Connector and Manual Policy Rules

The input layer uses a simulated SIEM connector rather than a live integration. The pluggable connector interface supports real integrations, but this has not been tested against an actual deployment. Similarly, the policy engine's rules—severity thresholds, entity-presence requirements, irreversibility constraints—are manually configured and require domain expertise to set correctly. There is no mechanism to learn or refine thresholds from operational outcomes, and appropriate values will differ substantially across environments. Operators must review and calibrate policy configuration before enabling autonomous execution.

8.3.4 Partially Implemented Feedback Loop

The system stores analyst feedback in the audit log and provides infrastructure for online learning, but the weight update mechanism is not fully integrated into the production inference path. Retraining currently requires an offline cycle using the accumulated feedback data.

8.4 Threats to Validity

8.4.1 Internal and External Validity

The main threat to internal validity is circularity between training and test data. Both sets are generated by the same synthetic pipeline, parameterised by the same domain-knowledge priors. A model that has learned the generator's internal structure will score well on the test set without necessarily having learned a generalisable response policy. Holding out 10% of examples with a fixed seed ensures the test examples were not seen during training, but it does not eliminate the underlying structural similarity between the partitions.

Generalisation to real SOC distributions or adversarial inputs is not established. Sommer and Paxson [10] identify distribution shift as a fundamental challenge for machine-learning-based security systems: attackers adapt in response to deployed defences, producing novel patterns outside the training distribution.

8.4.2 Construct and Statistical Validity

Strategy accuracy and action F1 measure label agreement with synthetic ground-truth labels. They do not measure operational effectiveness—whether the system actually reduces analyst workload or improves response times in a live SOC is not evaluated. This gap between model accuracy and operational impact is a recurring methodological problem in security automation research [2, 3].

All reported results are from single training runs with a fixed random seed. No variance estimates or confidence intervals are reported. Reproducibility is maintained: all training scripts, dataset generation pipelines, and model artefacts are committed to the repository with fixed seeds, enabling independent replication from first principles.

Conclusion and Future Work

9.1 Summary of Contributions

This thesis started from a specific observation: there is a gap between detecting a security threat and doing something useful about it. SIEM platforms handle detection well. SOAR platforms can execute predefined responses. But the reasoning layer in between—where a system evaluates what happened, decides what to do, and acts safely—is largely missing. The system built in this thesis is that layer.

9.1.1 Contributions of This Work

A layered, interface-driven orchestration architecture. Every stage in the pipeline—ingestion, normalization, enrichment, scoring, planning, policy evaluation, execution, and audit—sits behind a versioned interface. Any component can be replaced without touching the rest. This was deliberate: security tooling changes quickly, and the system has to accommodate that without requiring a full rewrite when a better scorer or connector comes along. The interface-driven design also makes meaningful unit testing feasible, which is harder than it sounds in security tooling where integration complexity often makes isolated coverage impractical.

Explainability as a data-model constraint, not a reporting feature. Every object that carries a decision also carries the reasoning behind it. A `ThreatAssessment` includes a hypothesis and an evidence list. A `DecisionPlan` includes per-action rationale strings. A `PolicyDecision` records the specific reason each action was approved, held, or denied. These fields are written at decision time and stored verbatim in the audit log—no post-hoc reconstruction, no secondary explanation module. This is a meaningful difference from the common pattern of training a model for accuracy and bolting on an explanation layer afterward.

Structured response prediction with deterministic entity resolution. Response planning was split into three structured classification problems—strategy selection, action set prediction, and priority regression—rather than treated as a generation task. The reason became clear from experiment: generative LLM-based planners hallucinate entity values, producing plans that reference hostnames and IP addresses that were never in the alert. A plan targeting the wrong resource is worse than no plan at all. By separating *what to do* (learned by the model) from *what to target* (resolved deterministically from the enriched alert), the system eliminates this failure mode entirely. The resulting custom neural network achieves 96.4% strategy accuracy and 97.2% action F1 in full pipeline evaluation with roughly 270 thousand parameters—matching a 250-million-parameter fine-tuned transformer at a fraction of the inference cost.

A policy engine that is structurally prior to execution. The policy layer is not advisory. It is a synchronous, in-process gate that every planned action must pass before dispatch. Hard denials—actions requiring entity data that is absent, irreversible actions on production assets below the severity threshold, globally disabled action types—are evaluated before any approval gate. There is no code path by which a model error or adversarial input can cause a hard-denied action to execute. This is a stronger guarantee than playbook-based SOAR platforms provide, where safety depends entirely on whoever authored the playbook.

A training pipeline that documents iterative improvement under real constraints. The dataset and training chapter documents not just the final model but the full path to it: from a zero-shot LLM baseline that produced passive-only responses for every alert, through QLoRA fine-tuning of a foundation security model that hallucinated entity values, through multiple seq2seq variants, to the final structured prediction approach. Each iteration exposed a specific failure mode and motivated a specific fix. The outcome is not just a trained model but a documented methodology for building reliable, safety-constrained response planners from synthetic data when real incident labels are unavailable.

An end-to-end runnable system with full observability. The system runs on a single development machine without cloud dependencies. Every alert processed during a session produces a demo trace—a complete snapshot of every stage it passed through—that can be replayed, inspected, and visualised. The console output during live processing shows every decision in real time: enrichment context, assessment score, chosen strategy, and the per-action policy outcome with specific reasons. This transparency makes the system demonstrable, debuggable, and auditable in a way that vendor black-box add-ons typically are not.

9.1.2 The Broader Argument

The response gap in security operations is not primarily a capability problem—it is a design problem. The capability to detect threats exists. The capability to execute responses exists. What has been missing is a principled intermediate layer that reasons about the connection between them, communicates that reasoning clearly, and enforces hard safety constraints independently of the reasoning quality. The system in this thesis is that layer. The evaluation is on synthetic data and the SIEM connectors are simulated, but the architecture is sound, the safety guarantees are structural, and the planning quality is high enough to show that learned response planning is practical and measurable—not a research curiosity.

9.2 Future Research Directions

The most urgent next step is evaluation on real SOC data. The results in this thesis reflect performance on synthetic test examples drawn from the same generator as the training data—a controlled setting that cannot replicate the distribution of a live environment, the edge cases analysts actually encounter, or the way attacker behaviour shifts over time. Even a small annotated set from an operational SOC would either validate the approach or expose failure modes the synthetic generator masks. On the model side, the intelligence layer would benefit from continual-learning safeguards—drift detection, bounded weight updates, periodic recalibration—to prevent gradual degradation as alert distributions shift. The policy engine currently relies on manually authored rules; learning or refining policy thresholds from analyst feedback would be a natural improvement once the feedback loop is fully integrated.

The most impactful evaluation, though, would be a deployment study with real SOC analysts. Offline accuracy metrics cannot measure whether the system actually reduces workload, improves response times, or earns analyst trust under realistic investigation pressure. A longitudinal study covering a full analyst shift cycle is the necessary next step toward making the reasoning layer something that can be deployed with confidence in a production environment.

References

[1] Karen Kent and Murugiah Souppaya. Guide to computer security log management. Technical Report NIST SP 800-92, National Institute of Standards and Technology, 09 2006. URL https://csrc.nist.gov/pubs/sp/800/92/final.

[2] Tao Ban, Takeshi Takahashi, Samuel Ndichu, and Daisuke Inoue. Breaking alert fatigue: Ai-assisted siem framework for effective incident response. *Applied Sciences*, 13(11):6610, 2023. doi: 10.3390/app13116610. URL https://www.mdpi.com/2076-3417/13/11/6610.

[3] J. Tilbury and S. Flowerday. Humans and automation: Augmenting security operation centers. *Journal of Cybersecurity and Privacy*, 4(3):388–409, 2024. doi: 10.3390/jcp4030020. URL https://www.mdpi.com/2624-800X/4/3/20.

[4] S. Sarhan, M. Haque, M. Asad, P. Kabo, A. Apon, and I. M. Elfadel. Rulegenie: Siem detection rule set optimization. *arXiv preprint arXiv:2505.06701*, 2025. doi: 10.48550/arXiv.2505.06701. URL https://arxiv.org/abs/2505.06701.

[5] Paul Cichonski, Tom Millar, Tim Grance, and Karen Scarfone. Computer security incident handling guide. Technical Report NIST SP 800-61 Rev. 2, National Institute of Standards and Technology, 2012. URL https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf.

[6] Fredrik Valeur, Giovanni Vigna, Christopher Kruegel, and Richard A. Kemmerer. A comprehensive approach to intrusion detection alert correlation. *IEEE Transactions on Dependable and Secure Computing*, 1(3):146–169, 2004. doi: 10.1109/TDSC.2004.21.

[7] Blake E. Strom, Andy Applebaum, Doug P. Miller, Kathryn C. Nickels, Adam G. Pennington, and Cody B. Thomas. MITRE ATT&CK: Design and philosophy. Technical Report MTR180002, The MITRE Corporation, 2018. URL https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf.

[8] Stefan Axelsson. The base-rate fallacy and the difficulty of intrusion detection. *ACM Transactions on Information and System Security*, 3(3):186–205, 2000. doi: 10.1145/357830.357849.

[9] Pedro Garcia-Teodoro, Jesus Diaz-Verdejo, Gabriel Macia-Fernandez, and Enrique Vázquez. Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security*, 28(1–2):18–28, 2009. doi: 10.1016/j.cose.2008.08.003.

[10] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, pages 305–316. IEEE, 2010. doi: 10.1109/SP.2010.25.

[11] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1285–1298. ACM, 2017. doi: 10.1145/3133956.3134015.

[12] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009. doi: 10.1109/TNN.2008.2005605.

[13] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[14] Roberto Rodriguez and Jose Luis Rodriguez. OTRF security datasets: Structured attack telemetry for security research. GitHub repository, 2020. URL https://github.com/OTRF/Security-Datasets.

[15] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116, 2018. doi: 10.5220/0006639801080116.

[16] Defense Advanced Research Projects Agency. Transparent computing (TC) program. DARPA Information Innovation Office, 2019. URL https://www.darpa.mil/research/programs/transparent-computing.

[17] Paul Kassianik, Baturay Saglam, Alexander Chen, et al. Llama-3.1-FoundationAI-SecurityLLM-base-8b technical report. arXiv preprint arXiv:2504.21039, 2025. URL https://arxiv.org/abs/2504.21039.

[18] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL https://arxiv.org/abs/2305.14314.

[19] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

[20] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://jmlr.org/papers/v12/pedregosa11a.html.

[21] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024. URL https://arxiv.org/abs/2210.11416.

[22] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://arxiv.org/abs/1912.01703.

Appendix

A.1 Additional Figures

A.2 Configuration Details